AI Service Desk Front Door — Implementation Playbook

Oct 2, 2026 · @Dewanshu

This playbook turns the approved FINAL BUILD CONTRACT into an ordered, task-by-task build plan for Claude Code. The contract remains the source of truth for *what* is built; this playbook is the source of truth for *how*. Anything not in the contract is marked **OPTIONAL** and is not built unless approved.

Stack alignment with the Build Contract

The mandatory stack replaces the contract's *recommended* stack (which the contract allowed). Record this as ADR-001 in `docs/DECISIONS.md` on day 1.

Contract said	Now	Consequence
FastAPI (Python) backend	Node.js + Express + TypeScript	AI, evals and API all in TypeScript; one language end to end
PostgreSQL + pgvector + full-text	MongoDB Atlas + Atlas Vector Search + Atlas Search	Hybrid search = `$vectorSearch` + `$search`, fused in code
Deterministic state machine	LangGraph `StateGraph`	Same design, expressed as graph nodes and conditional edges
Single LLM provider	Gemini or OpenRouter via config	Provider factory in LangChain; prompts stay provider-neutral
Docker Compose: web, api, db	Docker Compose: web, api, `mongodb-atlas-local`	Local Atlas image provides Search and Vector Search; Atlas cloud is the alternative

How LangGraph is used without breaking the contract:

- **One graph, not multiple agents.** The contract forbids multi-agent architecture. LangGraph runs a single `StateGraph` whose routing is decided by deterministic code (conditional edges reading state), not by an LLM choosing the next agent.

- **Tool execution is graph-owned.** The contract says the AI may call no tool other than retrieval. The LLM is never bound to tools. The graph itself executes two "tool nodes": retrieval and ticket creation. Ticket creation runs only after a human-in-the-loop `interrupt()` is resumed by the employee's confirmation.
- **State persists via a MongoDB checkpointer** (`thread_id` = conversation ID), so the ticket-draft interrupt survives across HTTP requests.

Decisions that need your approval before week 2 (the playbook assumes the defaults):

#	Decision	Default assumed	Why it needs approval
D-1	Reranker	LLM-based listwise rerank with a fast model, combined with the context-sufficiency verdict in one call	Neither Gemini nor OpenRouter offers a dedicated reranker in this stack; a dedicated rerank API would add a provider
D-2	Embedding provider	Gemini embeddings, configured separately from the chat model	Do not assume OpenRouter serves embeddings; changing the embedding model requires full re-index and a new vector index dimension
D-3	Local database	`mongodb-atlas-local` Docker image for dev and integration tests; Atlas free tier as fallback	Plain MongoDB and in-memory test servers do not support `$vectorSearch` / `$search`
D-4	Hybrid fusion	Reciprocal Rank Fusion in application code	Portable and testable; avoids depending on server-side fusion stages whose availability varies by version
D-5	DB access	Official MongoDB Node driver + Zod (no Mongoose)	Avoids two validation systems; LangChain's MongoDB integration uses the native driver

Product names, library packages and version-specific features above are from knowledge as of mid-2026; Claude Code must verify current package names and APIs at install time and record versions in the lockfile.

1. Implementation strategy

Build bottom-up along the data path, and put measurement before intelligence. Every AI stage is built only after the thing that measures it exists, so quality is known at each step rather than discovered in week 4.

Adjusted dependency chain:

1. **Foundation** — monorepo, TypeScript configs, shared Zod contracts, env and YAML config loading, logging, error envelope, Docker Compose.
2. **Database** — Mongo client, collections, indexes, repositories, counters.
3. **Authentication** — mock login, JWT cookie, RBAC middleware, audit service, frontend shell with route guards.
4. **LLM provider layer** *(moved up)* — chat and embedding model factory; ingestion needs embeddings.
5. **Corpus + golden set authoring** *(new, parallel)* — documents and eval cases are written together so retrieval can be measured immediately.
6. **Knowledge ingestion** — parse, chunk, embed, store, index.
7. **Retrieval** — access filter, vector + keyword search, fusion.
8. **Evaluation (retrieval)** *(split)* — recall@5 runner. Gate: ≥ 75%.
9. **AI orchestration** — prompt registry, LangGraph state and nodes, rerank, decision, answer, citation validation, persistence.
10. **Employee UI** — chat streaming, citations, source viewer, feedback, resolution.
11. **Evaluation (decisions)** — full-graph eval runner and stored runs.
12. **Ticketing** — taxonomy, routing, priority floors, slot filling, human-in-the-loop draft, ticket creation, My tickets.
13. **Agent workspace** — queue, detail, AI summary and draft reply, corrections.
14. **Admin console** — metrics, knowledge gaps, traces, documents, evaluation page.
15. **Evaluation (judge + tuning)** *(split)* — groundedness judge, threshold sweep.
16. **Hardening** — security, performance, resilience, seed data.
17. **Demo** — E2E suite for all nine flows, README, rehearsal.

Why these changes from your proposed order:

- **Evaluation is split into three passes** (retrieval, decisions, judge). One big evaluation phase would come too late to tune thresholds.
- **The LLM provider layer moves before ingestion**, because chunk embeddings need it.

- **Corpus and golden set are their own early track.** They are the largest non-code effort and block retrieval measurement.
- **Testing is continuous, not a phase.** Every task ships with its tests; the late "Testing" step becomes the E2E suite plus acceptance verification.
- **The frontend shell arrives with auth**, so every later feature has a place to appear.
- **Ticketing domain rules** (routing, priority floors) are pure functions with unit tests and could be built any time; they are scheduled in week 3 with the flow that uses them.

Build rules that apply to every phase:

- Thin vertical slices: API, AI and UI for one flow before starting the next flow.
- No AI call is added without a Zod output schema, a versioned prompt and a trace entry.
- No endpoint is added without a Zod request schema, RBAC rule and a Supertest test.
- Anything that must be exact (permissions, IDs, routing, priority floors, citations) is plain TypeScript with unit tests, never inside a prompt.

2. Repository architecture

A modular monolith in one repository using npm workspaces: two apps (web, api) and one shared package. The AI layer lives inside the api app as its own module boundary, so it can be extracted later without being a separate service now.

```
/
├── CLAUDE.md                  # Standing instructions for Claude Code
(rules, commands, links)
├── docs/
|   ├── BUILD_CONTRACT.md      # Approved contract (read-only)
|   ├── PLAYBOOK.md            # This document
|   ├── ARCHITECTURE.md        # Living architecture summary
|   ├── DECISIONS.md           # ADR log
|   ├── PROGRESS.md            # Task status, checkpoint results
|   └── PRODUCTION_GAPS.md
├── apps/
|   ├── web/                   # Next.js App Router frontend
|   |   └── src/
|   |       ├── app/           # Routes: (auth)/login, (employee)/chat,
tickets; (agent)/queue; (admin)/...
|   |       ├── components/    # ui/ (primitives), chat/, tickets/, agent/,
admin/
|   |       ├── lib/           # api client, SSE reader, auth helpers,
```

```
formatters
|   |       └── hooks/
|   └── api/                    # Express + TypeScript backend
|       └── src/
|           ├── server.ts / app.ts
|           ├── config/         # env schema, YAML loaders
|           ├── http/           # middleware (auth, rbac, errors, rate limit,
request id), router mount
|           ├── modules/        # auth, users, conversations, chat, tickets,
agent, documents, metrics, gaps, evals, audit
|           |   └── <module>/   # routes.ts, controller.ts, service.ts,
repository.ts, schemas.ts, *.test.ts
|           ├── ai/
|           |   ├── graph/      # state.ts, graph.ts (StateGraph wiring),
edges.ts, checkpointer.ts
|           |   ├── nodes/      # one file per node
|           |   ├── llm/        # provider factory (Gemini, OpenRouter),
embeddings factory, cost table
|           |   ├── prompts/    # registry.ts + <prompt>/v1.md ...
|           |   ├── schemas/    # Zod schemas for every LLM structured output
|           |   ├── retrieval/  # accessFilter, vectorSearch, keywordSearch,
fusion, rerank
|           |   ├── ingestion/  # loaders, chunker, embedder, indexer
|           |   ├── guards/     # citation validator, numeric check, pre-check
rules, redaction
|           |   └── tracing/    # trace builder, timers, token accounting
|           ├── domain/         # pure rules: routing, priority floors, ticket
schemas, decision rules
|           ├── db/             # client, collections, index definitions,
migrations
|           └── lib/            # logger, errors, ids, time
├── packages/
|   └── shared/                 # Zod API contracts, enums, DTO types shared
by web and api
├── config/                     # taxonomy.yaml, routing.yaml, priority-
rules.yaml, sensitive.yaml,
|                               # thresholds.yaml, ticket-schemas.yaml,
prompts.yaml, models.yaml
├── corpus/                     # it/, hr/ Markdown policies with YAML
frontmatter; corpus/README.md
├── evals/
|   ├── golden/                 # golden.v1.jsonl, schema notes
|   ├── runner/                 # CLI entry, scorers, judge, sweep
```

```
|   └── reports/                # generated, git-ignored
├── seed/                       # users.json, historic conversations/tickets
fixtures
├── scripts/                    # create-indexes, ingest, seed, reset, eval
wrappers
├── e2e/                        # Playwright specs and fixtures
├── docker-compose.yml
└── .env.example
```

Folder	Purpose and what belongs there	Must NOT go there
`docs/`	Contract, playbook, ADRs, progress log, production gaps	Generated reports, code
`apps/web/src/app`	Route segments, layouts, page-level data loading, role guards	Business rules, direct DB or LLM access, secrets
`apps/web/src/components`	Presentational and interactive components grouped by area	API URLs hard-coded, fetch logic beyond hooks
`apps/web/src/lib`	Typed API client using `packages/shared` schemas, SSE stream reader	Duplicated type definitions
`apps/api/src/http`	Cross-cutting middleware and router mounting	Feature logic
`apps/api/src/modules/*`	One feature each: routes → controller (parse/validate) → service (logic) → repository (DB)	LLM calls (go through `ai/`), cross-module repository access
`apps/api/src/ai/graph`	LangGraph state definition, node wiring, conditional edges, checkpointer	Prompt text, DB queries other than via repositories
`apps/api/src/ai/nodes`	One node per file, pure function of state plus injected dependencies	Routing decisions belonging to `domain/` rules

Folder	Purpose and what belongs there	Must NOT go there
`apps/api/src/ai/llm`	The only place that constructs chat/embedding models	Prompts, business logic
`apps/api/src/ai/prompts`	Versioned prompt files and the registry	Inline prompts anywhere else
`apps/api/src/ai/schemas`	Zod schemas for every structured output	API request schemas
`apps/api/src/ai/retrieval`	Filters, searches, fusion, rerank	Permission decisions made by an LLM
`apps/api/src/ai/guards`	Deterministic safety: pre-check, citation and numeric validation, PII redaction	Anything that calls an LLM
`apps/api/src/domain`	Pure, framework-free rules with exhaustive unit tests	I/O of any kind
`apps/api/src/db`	Connection, typed collection accessors, index definitions	Feature queries (those live in module repositories)
`packages/shared`	Zod request/response schemas, enums, SSE event types	Node-only or React-only code
`config/`	Tunable behaviour as data; every file validated by Zod at startup	Secrets, environment-specific URLs
`corpus/`	Synthetic source documents with metadata frontmatter	Real company documents, generated chunks
`evals/`	Golden dataset, runner, scorers, judge, sweep	Unit tests for app code
`seed/`, `scripts/`	Idempotent data setup and operational CLIs	Logic not reusable from services (scripts call services)

Folder	Purpose and what belongs there	Must NOT go there
`e2e/`	Playwright specs for the nine contract flows	Unit or API tests

Tests live next to the code they test (`*.test.ts`) for unit and integration tests; API tests use Supertest under `apps/api/test/api/` ; AI workflow tests under `apps/api/test/ai/` use a fake LLM; E2E tests in `e2e/` .

3. Database design

MongoDB with one database (`servicedesk`). Every collection has a Zod schema in `apps/api/src/db/schemas` ; documents are validated on write in repositories. All IDs are `ObjectId` unless stated; all timestamps are UTC `Date` . Unbounded children (messages, chunks, events, traces) are separate collections, never embedded arrays.

Relationship map

Relationship	Cardinality	How stored
users → conversations	1 : N	`conversations.userId`
conversations → messages	1 : N	`messages.conversationId`
messages (assistant) → turn_traces	1 : 1	`turn_traces.messageId` (unique)
documents → chunks	1 : N	`chunks.documentId`
messages ↔ chunks (citations)	N : M	`messages.citations[].chunkId` array
conversations → tickets	1 : 0..1 in MVP	`tickets.conversationId` (unique, sparse)
users → tickets (requester)	1 : N	`tickets.requesterId`
users (agents) ↔ queues	N : M	`users.queues[]` string array
tickets → ticket_events	1 : N	`ticket_events.ticketId`
messages → feedback	1 : N (one per user)	`feedback.messageId` + unique (messageId, userId)

Relationship	Cardinality	How stored
eval_runs → eval_results	1 : N	`eval_results.runId`
documents ↔ countries / employment types	N : M	arrays on documents, denormalised onto chunks

users

Field	Type	Notes
_id	ObjectId	
email	string	unique, lowercase
name	string	
role	enum `employee` | `agent` | `admin`	
country	enum `US` | `UK`	
employmentType	enum `full_time` | `contractor`	
department	string	
jobTitle	string	
accessLevel	enum `standard` | `manager`	controls document access
queues	string[]	agents only; values from `config/routing.yaml`
isDemo	boolean	shown on login picker
createdAt	Date	

Indexes: `{email:1}` unique; `{role:1}`. Constraint: `queues` non-empty iff role = agent.

documents

Field	Type	Notes
_id	ObjectId	
slug	string	stable ID from frontmatter, e.g. `hr-parental-leave-us`
title, domain	string, enum `IT` | `HR`	
owner	string	knowledge owner name
version	string	e.g. `3.1`
effectiveDate	Date	
countries	enum[]	`US`, `UK`, or both
employmentTypes	enum[]	
accessLevel	enum	minimum level required
status	enum `active` | `superseded` | `inactive`	
supersededBy	ObjectId | null	
sourceType	enum `markdown` | `pdf`	
sourcePath / contentHash	string	hash detects changes on re-ingest
chunkCount	number	
ingestStatus	enum `pending` | `indexed` | `failed`	
ingestError	string | null	
createdAt, updatedAt	Date	

Indexes: `{slug:1, version:1}` unique; `{status:1, domain:1}`. Constraint: only one `active` version per slug (enforced in service).

chunks

Field	Type	Notes
_id	ObjectId	the citation ID
documentId	ObjectId	
ordinal	number	position in document
sectionPath	string[]	heading trail, e.g. `["Parental Leave","Eligibility"]`
text	string	chunk body
contextualText	string	title + section path + text; this is what is embedded and keyword-indexed
tokenCount	number	
embedding	number[]	dimension from `config/models.yaml`
embeddingModel	string	
domain, countries, employmentTypes, accessLevel, status, effectiveDate, docTitle, docSlug	denormalised from document	required for pre-filtering in search indexes

Indexes: `{documentId:1, ordinal:1}` unique. **Atlas Vector Search index** `chunks_vector` on `embedding` (cosine) with filter fields `countries`, `employmentTypes`, `accessLevel`, `status`, `domain`. **Atlas Search index** `chunks_text` on `contextualText` (standard analyser) plus filterable token fields for the same metadata. Constraint: chunk metadata must be re-synced whenever document status or audience changes.

conversations

Field	Type	Notes
_id, userId	ObjectId	

Field	Type	Notes
title	string	first question, truncated
status	enum `open` | `resolved` | `escalated` | `handed_off` | `abandoned`	
resolution	`{ confirmed: boolean, at: Date, messageId }` | null	from "Did this solve it?"
ticketId	ObjectId | null	
restricted	boolean	true after a sensitive handoff
lastDecision	enum (six decisions)	
turnCount	number	
createdAt, updatedAt	Date	

Indexes: `{userId:1, updatedAt:-1}`; `{status:1, createdAt:-1}`; `{restricted:1}`.

Also: `isEval` (boolean, default false) on conversations, messages, turn_traces and tickets created by the eval runner; every dashboard and metrics query filters `isEval: false`. Index `{isEval:1, createdAt:-1}` on conversations.

messages

Field	Type	Notes
_id, conversationId	ObjectId	
role	enum `user` | `assistant` | `system_event`	
content	string	final text as shown
decision	enum | null	assistant only
reasonCode	string | null	
citations	`{ marker, chunkId, documentId, docTitle, sectionPath, effectiveDate }[]`	validated only

Field	Type	Notes
uncovered	string[]	PARTIAL: what was not covered
ticketDraft	object | null	snapshot shown to employee
status	enum `complete` | `withdrawn` | `error`	
createdAt	Date	

Indexes: `{conversationId:1, createdAt:1}`; `{decision:1, createdAt:-1}`.

turn_traces

Field	Type	Notes
_id, messageId, conversationId, userId	ObjectId	messageId = assistant message
graphRunId	string	LangGraph run/thread checkpoint ID
redactedInput	string	user message after PII redaction
profileSnapshot	`{ country, employmentType, accessLevel, department }`	
precheck	`{ sensitiveHit, safetyHit, matchedRules[] }`	
analysis	object	turn-analysis output (Zod-validated)
retrieval	`{ query, filter, vector[{chunkId,score,rank}], keyword[...], fused[...], reranked[{chunkId,relevance}], coverage }`	
decision	`{ value, reasonCode, rulesFired[], thresholds }`	

Field	Type	Notes
output	`{ citationsProposed[], citationsValid[], citationsRejected[], numericViolations[], withdrawn }`	
llmCalls	`{ node, promptName, promptVersion, promptHash, provider, model, inputTokens, outputTokens, costUsd, latencyMs, retries, error }[]`	
stepLatencyMs	record<string, number>	per node
totalLatencyMs, ttftMs	number	
totalCostUsd	number	
errors	`{ node, code, message }[]`	
restricted	boolean	copies conversation flag
createdAt	Date	

Indexes: `{messageId:1}` unique; `{conversationId:1, createdAt:1}`; `{createdAt:-1}`; `{"decision.value":1, createdAt:-1}`. TTL: none in prototype (production: retention policy).

tickets

Field	Type	Notes
_id	ObjectId	
humanId	string	e.g. `IT-1042`, `HR-0031`, from `counters`
conversationId, requesterId	ObjectId	requester from auth
requesterSnapshot	`{ name, email, country, employmentType, department }`	
domain	enum `IT` | `HR`	

Field	Type	Notes
category, subcategory	string	from taxonomy
queue	string	from routing table
priority	enum `low` | `medium` | `high` | `critical`	final, after floors
aiUrgency	enum	AI proposal before floors
priorityFloorApplied	string | null	rule ID
status	enum `new` | `in_progress` | `waiting_on_employee` | `resolved` | `closed`	
summary	string	one line
agentBrief	string	AI agent-ready description
fields	record<string, string | "unknown">	category schema values
aiFilledFields	string[]	keys filled by AI and not edited by employee
articlesShown	`{ documentId, docTitle }[]`	
restricted	boolean	sensitive handoff
duplicateOf	ObjectId | null	
createdAt, updatedAt, resolvedAt	Date	

Indexes: `{humanId:1}` unique; `{conversationId:1}` unique sparse; `{queue:1, status:1, priority:-1, createdAt:1}`; `{requesterId:1, createdAt:-1}`; `{requesterId:1, category:1, status:1}` (duplicate check). Constraints: `restricted = true` ⇒ `queue = hr_case`; status transitions validated by a domain state machine.

ticket_events

Field	Type	Notes
_id, ticketId, actorId	ObjectId	actorId null for system
type	enum `created` | `reply` | `note` | `status_change` | `classification_correction` | `ai_draft_generated`	
visibility	enum `public` | `internal`	notes and corrections are internal
body	string | null	
field	string | null	for corrections: `category`, `subcategory`, `priority`
oldValue, newValue	string | null	
createdAt	Date	

Indexes: `{ticketId:1, createdAt:1}`; `{type:1, createdAt:-1}` (override rate).

feedback

Fields: `_id`, `messageId`, `conversationId`, `userId` (ObjectId), `rating` (enum `up` | `down`), `comment` (string ≤ 500, optional), `createdAt`. Indexes: `{messageId:1, userId:1}` unique (upsert); `{rating:1, createdAt:-1}`.

eval_runs

Fields: `_id`, `suite` (enum `retrieval` | `full` | `judge`), `datasetVersion` (string), `status` (enum `queued` | `running` | `completed` | `failed`), `configSnapshot` (thresholds, prompt versions, models, gitCommit), `caseCount`, `metrics` (record<metric, number>), `perCategory` (record<category, record<metric, number>>), `targets` (record<metric, {target, met}>), `passed` (boolean), `costUsd`, `durationMs`, `triggeredBy` (ObjectId | `cli`), `startedAt`, `finishedAt`, `error`. Indexes: `{startedAt:-1}`; `{suite:1, startedAt:-1}`. Constraint: at most one `running` run (checked in service).

eval_results

Fields: `_id`, `runId`, `caseId` (string), `category` (enum case category), `tags` (string[]), `expected` (object), `actual` (`{ decision, retrievedDocSlugs[], citedDocSlugs[], category, priority, fields, answerText }`), `scores` (record<metric, number | boolean>), `passed` (boolean), `failureReasons` (string[]), `traceRef` (object, inline trace summary), `latencyMs`, `costUsd`. Indexes: `{runId:1, caseId:1}` unique; `{runId:1, passed:1}`.

audit_log

Fields: `_id`, `actorId` (ObjectId | null), `actorRole`, `action` (string, e.g. `auth.login`, `ticket.status_change`, `document.upload`, `eval.run`, `admin.trace_view`), `targetType`, `targetId`, `metadata` (object, no PII beyond IDs), `ip` (string), `requestId`, `createdAt`. Indexes: `{createdAt:-1}`; `{actorId:1, createdAt:-1}`; `{targetType:1, targetId:1}`. Constraint: append-only; no update or delete routes exist.

Supporting collections

- **counters** — `{ _id: "IT" | "HR", seq: number }`, incremented atomically with `findOneAndUpdate` to generate `humanId`.
- **gap_clusters** — `{ _id, snapshotAt, label, domain, questionCount, exampleQuestions[] (redacted), messageIds[], centroid? }`; regenerated on demand; index `{snapshotAt:-1}`.
- **LangGraph checkpoints** — collections created and owned by the MongoDB checkpointer package; never read or written by application code directly.

4. API contract

Conventions for every endpoint

- Base path `/api`. JSON in and out, except the chat stream (SSE) and document upload (multipart).
- Every request and response body has a Zod schema in `packages/shared/src/api/<group>.ts`; the web client and the api both import it.
- Auth: httpOnly `sd_session` cookie containing the JWT. Roles: `employee`, `agent`, `admin`. "Any" means any authenticated role.

- Error envelope: `{ error: { code, message, details?, requestId } }`. Common codes: `VALIDATION_ERROR` 400, `UNAUTHENTICATED` 401, `FORBIDDEN` 403, `NOT_FOUND` 404, `CONFLICT` 409, `RATE_LIMITED` 429, `LLM_UNAVAILABLE` 503, `INTERNAL` 500. Every endpoint can return 400/401/500; only extra cases are listed below.
- Resources owned by other users return `NOT_FOUND`, not `FORBIDDEN`, to avoid revealing existence.
- Lists are paginated with `?limit` (default 20, max 100) and `?cursor`; responses include `nextCursor`.
- Every mutating call writes an `audit_log` entry where marked "audit".
- `GET /api/health` (public) returns `{ status, db, llmConfigured }`; no DB effect.

Auth

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`GET /api/auth/demo-users`	Public	—	`{ users: [{ id, name, role, country, employmentType, jobTitle }] }`	—	Read users where `isDemo`	No
`POST /api/auth/login`	Public	`{ userId }` or `{ email }` (+ `DEMO_PASSWORD` if env set)	`{ user }` + sets cookie	401 unknown user / bad demo password; 429	audit `auth.login`	No
`POST /api/auth/logout`	Any	—	204, clears cookie	—	audit `auth.logout`	No
`GET /api/auth/me`	Any	—	`{ user }`	401	Read	No

Profiles

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`GET /api/profiles/me`	Any	—	`{ name, email, role, country, employmentType, department, jobTitle, accessLevel, queues? }`	401	Read	No

Profiles are read-only in the MVP; they come from seed data (production: HRIS).

Conversations

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`POST /api/conversations`	Employee	`{}`	`{ conversation }`	429	Insert conversation	No
`GET /api/conversations`	Employee	`?limit&cursor&status`	`{ items: [{ id, title, status, lastDecision, updatedAt, ticketHumanId? }], nextCursor }`	—	Read own	No
`GET /api/conversations/:id`	Employee (owner)	—	`{ conversation, messages: [...], pendingDraft?: TicketDraft }`	404	Read; pending draft read from graph checkpoint state	No

Chat

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`POST /api/conversations/:id/messages`	Employee (owner)	`{ content: string (1-2,000 chars) }`	`text/event-stream` (events below)	404; 409 if a turn is already running or a ticket draft awaits confirmation; 429; stream `error` event for LLM failures	Insert user message; graph run; insert assistant message; insert turn_trace; update conversation	**Yes**: full graph
`POST /api/conversations/:id/escalate`	Employee (owner)	`{ reason?: string }`	SSE, same events	404; 409	Same as above, with `forceEscalate` in graph input	Yes: slot fill
`POST /api/conversations/:id/resolution`	Employee (owner)	`{ messageId, resolved: boolean }`	`{ conversation }`	404; 409 if message is not ANSWER/PARTIAL	resolved=true → status `resolved`, `resolution` set; false → system_event message offering escalation	No
`POST /api/messages/:id/feedback`	Employee (owner)	`{ rating: "up" | "down", comment? }`	`{ feedback }`	404; 409 if not an assistant message	Upsert feedback	No
`GET /api/sources/chunks/:chunkId`	Employee	—	`{ chunk: { id, text, sectionPath }, document: { title, version, effectiveDate, owner }, neighbours: [prev, next] }`	404 if chunk not accessible to caller's profile	Read; access filter re-applied	No

SSE event types (schema in `packages/shared/src/api/chat-events.ts`):

Event	Payload	When
`turn_started`	`{ userMessageId, assistantMessageId }`	Immediately
`status`	`{ stage: "analyzing" | "searching" | "writing" | "preparing_ticket" }`	Stage changes, for UI progress
`token`	`{ text }`	ANSWER/PARTIAL/CLARIFY text streaming
`citations`	`{ items: Citation[] }`	After validation, before `done`
`withdrawn`	`{ reason, replacementText }`	Citation validation failed; UI replaces streamed text

Event	Payload	When
`ticket_draft`	`{ draft: TicketDraft, missingFields: string[] }`	ESCALATE reached draft stage
`handoff`	`{ kind: "sensitive" | "out_of_scope", text, contact? , ticketHumanId? }`	Templated responses
`done`	`{ decision, reasonCode, assistantMessageId, latencyMs }`	Always last on success
`error`	`{ code, message, retryable }`	Failure; message persisted with status `error`

Tickets (employee)

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`POST /api/conversations/:id/ticket-draft/confirm`	Employee (owner)	`{ edits: record<field,string>, duplicateDecision?: "new" | "same" }`	`{ ticket: { id, humanId, queue, priority, status } }`	404 no pending draft; 409 `DUPLICATE_CANDIDATE` with `{ existing: { humanId, summary, status } }` when no `duplicateDecision` given; 400 edits fail category schema	Resumes graph interrupt → createTicket node: counter increment, insert ticket, insert `created` event, update conversation, system message; audit `ticket.create`. `same` → links to existing, no new ticket	Yes: agent summary generated after creation
`POST /api/conversations/:id/ticket-draft/cancel`	Employee (owner)	—	`{ conversation }`	404 no draft	Resumes graph with cancel; system message	No
`GET /api/tickets/mine`	Employee	`?status&limit&cursor`	`{ items: [{ id, humanId, summary, status, priority, updatedAt }] }`	—	Read own	No
`GET /api/tickets/:id`	Employee (requester)	—	`{ ticket (employee view: no internal fields), events: public only }`	404	Read	No
`OPTIONAL: POST /api/tickets/:id/replies`	Employee (requester)	`{ body }`	`{ event }`	404; 409 if closed	Insert public reply; status `waiting_on_employee` → `in_progress`	No

`TicketDraft` = `{ domain, category, subcategory, proposedPriority, summary, description, fields: record<string,string|"unknown">, requiredMissing: string[], safetyNotice? }`. Requester, queue and final priority are not in the draft; code sets them on confirm.

OPTIONAL: `POST /api/tickets/:id/replies` (employee replies on a ticket) is the contract's stretch goal; build it only if week 3 finishes early.

Agent

Agents see only tickets whose `queue` is in their `users.queues`. Restricted tickets are visible only to `hr_case` agents and admins. Admins may call agent endpoints for any queue.

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`GET /api/agent/tickets`	Agent, Admin	`?queue&status&priority&category&limit&cursor`	`{ items: [{ id, humanId, summary, priority, status, category, queue, requesterName, ageMinutes, restricted }] }` sorted priority desc, then createdAt asc	403 queue not assigned	Read	No
`GET /api/agent/tickets/:id`	Agent (queue), Admin	—	`{ ticket (full), aiFilledFields, articlesShown, events (all), transcript: messages[], citationsShown }`	404	Read; audit `ticket.view` for restricted	No
`POST /api/agent/tickets/:id/draft-reply`	Agent (queue), Admin	`{ tone?: "standard" | "brief" }`	`{ draft: { body, citedDocuments[] }, promptVersion }`	404; 503	Insert `ai_draft_generated` internal event; LLM call metadata stored on event	**Yes**: agent draft reply
`POST /api/agent/tickets/:id/replies`	Agent (queue), Admin	`{ body, usedAiDraft: boolean }`	`{ event }`	404; 409 if closed	Insert public reply event; audit	No
`POST /api/agent/tickets/:id/notes`	Agent (queue), Admin	`{ body }`	`{ event }`	404	Insert internal note	No
`PATCH /api/agent/tickets/:id/status`	Agent (queue), Admin	`{ status }`	`{ ticket }`	404; 409 invalid transition	Update ticket; insert `status_change` event; audit	No
`PATCH /api/agent/tickets/:id/classification`	Agent (queue), Admin	`{ category?, subcategory?, priority? }` (at least one)	`{ ticket }`	404; 400 not in taxonomy	Update ticket; one `classification_correction` event per changed field with old/new; re-route queue if category changes; audit	No

Admin

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`GET /api/admin/conversations`	Admin	`?decision&status&from&to&userId&limit&cursor`	`{ items: [{ id, userName, title, status, lastDecision, turnCount, restricted, createdAt }] }`	—	Read	No
`GET /api/admin/conversations/:id/traces`	Admin	—	`{ conversation, turns: [{ userMessage (redacted), assistantMessage, traceSummary }] }`	404	Read; audit `admin.trace_view`	No
`GET /api/admin/traces/:traceId`	Admin	—	`{ trace }` full (redacted input) with chunk texts resolved	404	Read; audit	No
`GET /api/admin/knowledge-gaps`	Admin	`?snapshot=latest`	`{ snapshotAt, clusters: [{ id, label, domain, questionCount, exampleQuestions[] }] }`	404 if no snapshot	Read gap_clusters	No
`POST /api/admin/knowledge-gaps/refresh`	Admin	`{ from?, to? }`	`{ snapshotAt, clusterCount }` (synchronous, ≤ 30 s)	409 refresh running; 503	Read low-evidence turns (non-restricted), cluster embeddings, insert gap_clusters snapshot; audit	**Yes**: cluster labelling
`GET /api/admin/audit-log`	Admin	`?action&actorId&from&to&limit&cursor`	`{ items }`	—	Read	No

Documents

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`GET /api/admin/documents`	Admin	`?domain&status&country`	`{ items: [{ id, slug, title, domain, version, status, countries, employmentTypes, accessLevel, effectiveDate, chunkCount, ingestStatus }] }`	—	Read	No
`GET /api/admin/documents/:id`	Admin	—	`{ document, chunks: [{ id, ordinal, sectionPath, tokenCount, preview }] }`	404	Read	No
`POST /api/admin/documents`	Admin	multipart: `file` (.md or .pdf, ≤ 5 MB) + `metadata` JSON (slug, title, domain, owner, version, effectiveDate, countries, employmentTypes, accessLevel, supersedes?)	`{ document }` with `ingestStatus`	400 bad type/size/metadata; 409 slug+version exists	Insert document; chunk, embed, insert chunks; if `supersedes`, mark old version superseded and re-sync its chunks; audit	Embeddings only

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`POST /api/admin/documents/:id/reindex`	Admin	—	`{ document }`	404; 409 already indexing	Delete and recreate chunks; audit	Embeddings only
`PATCH /api/admin/documents/:id/status`	Admin	`{ status: "active" | "inactive" | "superseded" }`	`{ document }`	404; 409 would create two active versions	Update document and denormalised chunk status; audit	No

Evaluation

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`POST /api/admin/evals/runs`	Admin	`{ suite: "retrieval" | "full" | "judge", tags?: string[], caseIds?: string[] }`	202 `{ runId }`	409 a run is already running	Insert eval_run `queued`; in-process background job runs cases, writes eval_results, updates run; audit	**Yes**: runs the graph; judge suite uses judge prompt
`GET /api/admin/evals/runs`	Admin	`?suite&limit&cursor`	`{ items: [{ id, suite, status, passed, metrics, caseCount, startedAt, durationMs, costUsd }] }`	—	Read	No
`GET /api/admin/evals/runs/:id`	Admin	—	`{ run (metrics, perCategory, targets, configSnapshot) }`	404	Read	No
`GET /api/admin/evals/runs/:id/results`	Admin	`?passed=false&category&limit&cursor`	`{ items: [{ caseId, category, passed, failureReasons, expected, actual, scores }] }`	404	Read	No

The same runner is invoked by CLI (`npm run eval -- --suite full`), writing the same collections with `triggeredBy: "cli"`.

Metrics

Endpoint	Auth	Request	Response	Errors	DB effect	AI
`GET /api/admin/metrics/overview`	Admin	`?from&to` (default last 7 days)	`{ conversations, decisionDistribution, confirmedResolutionRate, escalationRate, escalationByCategory: [{ category, count, rate }], sensitiveHandoffs, overrideRate, thumbsDownRate, latency: { ttftP50, ttftP90, totalP50, totalP90 }, costPerConversationUsd }`	400 invalid range	Aggregation pipelines over conversations, messages, turn_traces, tickets, ticket_events, feedback	No

Metric definitions (fixed, documented in code): confirmed resolution rate = conversations with `resolution.confirmed = true` ÷ conversations with at least one ANSWER/PARTIAL; escalation rate = conversations with a ticket ÷ all conversations; override rate = tickets with ⩾ 1 classification correction ÷ tickets created; thumbs-down rate = down ÷ all feedback.

5. AI pipeline design

One compiled LangGraph `StateGraph` (`supportTurnGraph`) handles every employee turn. It uses a MongoDB checkpointer with `thread_id = conversationId`, so a ticket draft paused by `interrupt()` resumes on the confirm request. Routing between nodes is done by deterministic edge functions in `ai/graph/edges.ts` that read state; no LLM chooses the next node.

Graph state (`ai/graph/state.ts`, Zod-validated at node boundaries)

Key	Type	Written by
`input`	`{ conversationId, userId, userMessageId, assistantMessageId, text, forceEscalate }`	API before invoke
`profile`	`{ country, employmentType, accessLevel, department, jobTitle }`	loadContext

Key	Type	Written by
`history`	last 8 messages `{ role, content }`	loadContext
`ticketFlow`	`{ active, category?, clarifyRounds, fields, requiredMissing }`	slotFill / persisted across turns by checkpointer
`precheck`	`{ sensitiveHit, safetyHit, matchedRules[] }`	precheck
`analysis`	`TurnAnalysis` (section 6, P-1)	analyzeTurn
`retrieval`	`{ query, filter, vector[], keyword[], fused[], reranked[], coverage, topRelevance }`	retrieve, rerankAssess
`decision`	`{ value, reasonCode, rulesFired[] }`	decide
`output`	`{ text, citations[], uncovered[], withdrawn, draft?, handoff? }`	answer / clarify / handoff nodes / validate
`trace`	`{ llmCalls[], stepLatencyMs, errors[] }` (append reducer)	every node

Node sequence and routing

1. `START → loadContext → precheck`
2. `precheck →` **sensitiveHandoff** if `sensitiveHit`; else `analyzeTurn`
3. `analyzeTurn →` edge `routeAfterAnalysis`:
   a. `analysis.sensitivity = sensitive` → **sensitiveHandoff**
   b. `ticketFlow.active` (employee answering a ticket clarification) or `forceEscalate` → **slotFill**
   c. otherwise → **retrieve**
4. `retrieve → rerankAssess → decide`
5. `decide →` edge `routeByDecision`: ANSWER/PARTIAL → **answer**; CLARIFY → **clarify**; ESCALATE with safety → **answer** (safety-steps mode); ESCALATE → **slotFill**; OUT_OF_SCOPE → **outOfScope**; SENSITIVE_HANDOFF → **sensitiveHandoff**
6. `answer → validateCitations →` edge: safety mode → **slotFill**; withdrawn → **persist** (with escalation offer); else **persist**

7. `slotFill →` edge: required fields missing and `clarifyRounds < 2` → **clarifyForTicket** → **persist**; else **buildDraft** → **persist** → **awaitConfirmation** (`interrupt()` with the draft)
8. On resume: `confirm → createTicket (tool node) → agentSummary → END`; `cancel → END` (ticketFlow reset)
9. `clarify`, `outOfScope`, `sensitiveHandoff` → **persist** → `END`

Decision rules (`domain/decision-rules.ts`, evaluated top to bottom, first match wins)

Order	Condition	Decision	Reason code
1	`precheck.sensitiveHit` or `analysis.sensitivity = sensitive`	SENSITIVE_HANDOFF	`SENSITIVE_TOPIC`
2	`analysis.intent = out_of_scope`	OUT_OF_SCOPE	`OUT_OF_SCOPE`
3	`precheck.safetyHit`	ESCALATE (safety mode)	`SAFETY`
4	`forceEscalate` or `intent = talk_to_human`	ESCALATE	`USER_REQUESTED`
5	`intent = request_action` (needs a human to act)	ESCALATE	`ACTION_REQUIRED`
6	`analysis.ambiguity = high` and no CLARIFY yet this conversation topic	CLARIFY	`AMBIGUOUS`
7	`topRelevance ⩾ T_answer` and `coverage = full`	ANSWER	`EVIDENCE_STRONG`
8	`topRelevance ⩾ T_partial` and `coverage ∈ {full, partial}`	PARTIAL	`EVIDENCE_PARTIAL`
9	otherwise	ESCALATE	`NO_EVIDENCE`

`T_answer`, `T_partial` live in `config/thresholds.yaml` and are tuned by the eval sweep (section 8). A request-action that also has a helpful policy (e.g. access requests needing manager approval) still escalates, but the draft includes the policy citation.

Stage specification

Stage	Owner	Input	Output schema	Failure behaviour	Logged in turn_traces
Input	API (code)	`{ content }`	`ChatMessageRequest` (1–2,000 chars)	400; 409 if turn running or draft pending	— (user message stored)
loadContext	Code	conversationId, userId	`profile`, `history`, `ticketFlow`	DB error → `error` event, retryable	`profileSnapshot`, latency
precheck	Code	text	`{ sensitiveHit, safetyHit, matchedRules[] }` from `config/sensitive.yaml` + `priority-rules.yaml`	Cannot fail (pure)	`precheck`
analyzeTurn	AI (P-1, fast model)	text, history, profile summary, taxonomy	`TurnAnalysis`	Zod fail → 1 retry → fallback `{ intent: question, domain: unknown, query: raw text, ambiguity: low, sensitivity: none }` and flag `ANALYSIS_FALLBACK`	output, llmCall, retries, fallback flag
retrieve	Code + embeddings	`analysis.searchQuery`, domain hint, `profile`	`{ filter, vector[20], keyword[20], fused[12] }` with scores	Embedding failure → keyword-only + flag; DB failure → `error` event	filter, all candidate IDs + scores, latency
rerankAssess	AI (P-2, fast model)	question, 12 fused chunks (IDs + text)	`RerankAssessment`	Zod fail → 1 retry → fallback: fused order, `relevance` from RRF scaled, `coverage: unknown` (rules 7–8 cannot fire)	reranked list, coverage, llmCall
decide	Code	precheck, analysis, retrieval, ticketFlow, thresholds	`{ value, reasonCode, rulesFired[] }`	Cannot fail (pure)	decision, thresholds snapshot
answer	AI (P-3, quality model, streamed)	question, profile summary, top 5 chunks labelled `[S1]…[S5]`, mode (`answer` | `partial` | `safety_steps`)	Streamed text with `[Sx]` markers, then trailing JSON block `{ uncovered: string[] }` parsed after stream	Stream error → message `error`, `error` event retryable	tokens, TTFT, llmCall
validateCitations	Code	answer text, marker → chunkId map, retrieved set	`{ citationsValid[], citationsRejected[], numericViolations[], withdrawn }`	Zero valid citations → `withdrawn = true`, replacement text offers escalation, decision recorded as ESCALATE/ `CITATION_INVALID`	all three lists
clarify	AI (P-4, fast)	question, ambiguity reason, history	`{ question: string }`	Fail → templated generic clarifying question	llmCall
slotFill	AI (P-5, quality)	full conversation, category schema from `ticket-schemas.yaml`, profile	`SlotFillResult`	Fail → all fields `unknown`, description = redacted transcript summary flag	fields, missing list
clarifyForTicket	AI (P-4 in ticket mode)	missing required fields	`{ question }` asking ≤ 2 fields	Fail → templated question listing missing fields	llmCall
buildDraft	Code	slotFill result, analysis, precheck	`TicketDraft`	Cannot fail	draft snapshot
awaitConfirmation	LangGraph `interrupt()`	draft	resume value `{ action: confirm | cancel, edits, duplicateDecision }`	Never resumed → conversation stays `open` with pending draft	interrupt timestamp

Stage	Owner	Input	Output schema	Failure behaviour	Logged in turn_traces
createTicket	Code (tool node)	confirmed draft, profile, routing and priority rules	`Ticket`	Duplicate → return `DUPLICATE_CANDIDATE` to API without creating; DB failure → 500, graph stays interrupted, confirm is retryable (unique `conversationId`)	ticket ID, queue, priority floor
agentSummary	AI (P-6, quality)	ticket fields, transcript, cited docs	`{ summary, agentBrief }`	Fail → keep draft summary, `agentBrief` = description; flag on ticket	llmCall
outOfScope / sensitiveHandoff	Code (templates)	decision, domain	handoff text, contact, restricted ticket for sensitive	Cannot fail except DB	decision
persist	Code	full state	assistant message, turn_trace, conversation update	DB failure → logged, `error` event; never leaves a message without status	—
Response	API (code)	stream of node events	SSE events (section 4)	Client disconnect → graph completes and persists anyway	total latency, cost

Sensitive handoff detail: no AI advice and no field extraction. The templated reply shows a human contact and asks "Would you like HR to contact you privately?". On Yes (a one-click confirm, which satisfies the contract's confirmation rule), a restricted ticket is created in `hr_case` with a template summary, and the conversation is marked `restricted`. The employee never reviews AI-extracted fields, which would be the wrong experience for a distressed person.

Streaming: the API consumes `graph.streamEvents` (or the custom stream writer) and maps node starts to `status` events and answer-node LLM tokens to `token` events. `[Sx]` markers stream as-is; the UI renders them as pending citation chips until the `citations` event arrives.

6. Prompt architecture

Every prompt is a LangChain `ChatPromptTemplate` loaded from a file in `apps/api/src/ai/prompts/<name>/v<N>.md`. Frontmatter declares `name`, `version`, `modelTier` (`fast` | `quality`), `outputSchema` (name of a Zod schema in `ai/schemas`) and `temperature`. The registry loads the active version from `config/prompts.yaml`, computes a content hash, and every LLM call records `name@version#hash` in its trace.

Rules shared by all prompts

- Provider-neutral wording; no provider-specific syntax.
- User text, history and retrieved documents are placed in clearly delimited data sections, with an explicit instruction that content inside them is data and never instructions.

- Structured outputs use `withStructuredOutput(zodSchema)`. If a provider or model lacks native structured output, the llm layer falls back to JSON mode plus Zod parsing. Either way: one retry with the validation error appended, then the node's documented fallback.
- Enums in schemas come from config (taxonomy, categories), injected into the prompt at render time, so prompts and validation never drift.
- No prompt receives data the user is not allowed to see (filtering happens before).

Versioning strategy: a behaviour change creates `v<N+1>.md`; old versions stay in the repo. Switching the active version is a one-line change in `config/prompts.yaml` and requires a `full` eval run whose result is noted in `docs/PROGRESS.md`. Eval runs store the active prompt versions in `configSnapshot`, so any score can be traced to exact prompt text.

ID	Prompt	Purpose	Input	Structured output (Zod)	Key rules	Failure handling
P-1	`turn_analysis`	Understand the turn and produce a search query	message, last 8 messages, profile summary (country, employment type, department), taxonomy	`TurnAnalysis { intent: question | report_issue | request_action | talk_to_human | follow_up | out_of_scope; domain: IT | HR | unknown; category; subcategory?; urgency: low..critical; urgencyReason; sensitivity: none | sensitive; sensitivityReason?; ambiguity: low | high; ambiguityReason?; searchQuery (standalone, ≤ 30 words) }`	Resolve pronouns from history into the search query; never answer the question; mark sensitive generously (false positives are acceptable)	Retry once → fallback analysis (section 5)
P-2	`rerank_assess`	Order candidates by relevance and judge whether they answer the question	question, profile summary, 12 chunks with IDs, titles, section paths, effective dates	`RerankAssessment { ranked: [{ chunkId, relevance: 0..3, reason (≤ 15 words) }], coverage: full | partial | none, missingAspects: string[] }`	Only IDs from the input; judge relevance for *this* employee's profile; `full` only if every part of the question is answered	Retry → fallback to fused order, `coverage: unknown`; unknown IDs dropped by code

ID	Prompt	Purpose	Input	Structured output (Zod)	Key rules	Failure handling
P-3	`grounded_answer`	Write the employee-facing answer	question, profile summary, top 5 chunks labelled `[S1]`–`[S5]`, mode	Streamed prose with `[Sx]` markers, then a fenced JSON trailer `{ uncovered: string[] }`	Use only the sources; every factual sentence carries a marker; numbers and dates copied exactly; in `partial` mode state plainly what is not covered; in `safety_steps` mode give only the policy's safety steps, then say a ticket is being prepared; ≤ 180 words; no speculation	Stream failure → `error`; missing/invalid trailer → `uncovered: []`; citation problems handled by validator
P-4	`clarifying_question`	Ask one targeted question	mode (`ambiguity` | `ticket_fields`), question, ambiguity reason or missing field names with descriptions	`{ question: string (≤ 40 words), fieldsAsked: string[] }`	One question; at most 2 fields; never ask for data already in profile or history	Retry → template
P-5	`ticket_slot_fill`	Extract ticket fields from the conversation	full conversation, category schema (fields, descriptions, required flags), analysis	`SlotFillResult { category, subcategory, urgency, urgencyReason, impactScope: me | team | org, summary (≤ 15 words), description (agent-ready, ≤ 120 words), alreadyTried: string[], errorMessages: string[], fields: record<field, string | "unknown">, requiredMissing: string[] }`	Never guess: absent information is `unknown`; error text verbatim; no requester identity fields (code adds them)	Retry → all `unknown`, template summary
P-6	`agent_ticket_summary`	Agent-facing summary at ticket creation	ticket fields, redacted transcript, cited documents	`{ summary (≤ 15 words), agentBrief (≤ 100 words), suggestedFirstChecks: string[] (≤ 3) }`	Factual, no blame, mention articles already shown	Keep draft summary
P-7	`agent_draft_reply`	Draft the agent's first reply	ticket, agentBrief, cited documents' chunks, tone	`{ body (≤ 150 words), citedDocumentIds: string[] }`	Only claims supported by the given docs or ticket; placeholders like `[confirm date]` where unknown; agent must edit before sending	503 to the agent UI with retry
P-8	`gap_cluster_label`	Name a cluster of unanswered questions	5–10 redacted example questions per cluster	`{ label (≤ 8 words), domain: IT | HR | unknown }`	Describe the missing topic, not the users	Label = "Unlabelled cluster"

ID	Prompt	Purpose	Input	Structured output (Zod)	Key rules	Failure handling
P-9	`eval_judge` (offline only)	Score groundedness and key facts	question, answer, cited chunk texts, expected key facts	`{ groundedness: fully | partially | unsupported, unsupportedClaims: string[], keyFactsPresent: record<fact, boolean> }`	Strict: any unsupported claim → not `fully`; uses the quality model, never the same call as generation	Case marked `judge_error`, excluded and counted

Not LLM prompts (templates in `config/templates.yaml`): out-of-scope reply, sensitive handoff reply, withdrawn-answer replacement, ticket-confirmed message, generic error message.

7. Retrieval architecture

Retrieval is profile-filtered hybrid search: Atlas Vector Search and Atlas Search run in parallel with the same access pre-filter, results are fused with Reciprocal Rank Fusion in code, re-checked against the profile, then reranked by the LLM. Permissions are enforced three times, never by the model.

Ingestion path (`ai/ingestion`, CLI `npm run ingest` and the upload endpoint share it)

1. **Load** — Markdown via file read; PDF via LangChain's PDF loader. Reject anything else.
2. **Parse metadata** — Markdown YAML frontmatter (or upload metadata JSON) validated by `DocumentMetadataSchema`: slug, title, domain, owner, version, effectiveDate, countries, employmentTypes, accessLevel, status, supersedes. Missing or invalid metadata fails ingestion; no defaults are guessed.
3. **Normalise** — strip frontmatter, normalise whitespace, keep tables as Markdown tables.
4. **Chunk** — heading-aware: split on H2/H3 first (LangChain Markdown header splitter), then a recursive character splitter for long sections. Target 350–500 tokens, overlap 60 tokens. A table is never split mid-row; a section with a table keeps its heading. PDFs: recursive splitter only, page number kept in `sectionPath`.
5. **Contextualise** — `contextualText = "<title> > <section path>\n<text>"`. This improves retrieval on short sections like "Eligibility".
6. **Embed** — embeddings factory (`ai/llm/embeddings.ts`), batch of 50, retry with backoff. Embedding model and dimension come from `config/models.yaml` and are stored on each chunk.

7. **Store** — in one operation per document: delete old chunks for that document, insert new chunks with denormalised metadata, update `documents.chunkCount` and `ingestStatus`. If `supersedes` is set, the old version and its chunks are set to `superseded`.
8. **Index** — Atlas indexes are defined in `db/indexes.ts` and created by `npm run db:indexes`; indexing is asynchronous in Atlas, so ingestion waits until a test query returns the new chunks (poll, 30 s timeout).

Idempotency: re-running ingest skips documents whose `contentHash` and metadata are unchanged.

Query path (`ai/retrieval`)

Step	What happens	Parameters (config)
1. Build access filter	`buildAccessFilter(profile)` → `status = active` AND `countries` contains profile.country AND `employmentTypes` contains profile.employmentType AND `accessLevel ∈ allowedLevels(profile.accessLevel)`; optional `domain` hint only if analysis is confident	—
2. Embed query	`analysis.searchQuery` embedded with the same model as chunks	—
3. Vector search	`$vectorSearch` on `chunks_vector` with the filter as **pre-filter**	`numCandidates 150`, `limit 20`
4. Keyword search	`$search` on `chunks_text`: `compound.must` text query on `contextualText`, `compound.filter` with the same access conditions	`limit 20`
5. Fuse	Reciprocal Rank Fusion in code: `score = Σ 1 / (k + rank)`, deduplicate by chunk ID	`k = 60`, keep top 12
6. Post-filter assertion	`assertAccessible(chunk, profile)` re-checks every fused chunk's metadata; any violation is dropped, logged as `ACCESS_FILTER_VIOLATION` and fails tests	—

Step	What happens	Parameters (config)
7. Rerank + assess	P-2 orders the 12 and returns `relevance 0–3` and `coverage`	`topRelevance` = max relevance
8. Final context	Top 5 by relevance with `relevance ⩾ 1`, grouped by document, labelled `[S1]–[S5]` with title, section path and effective date	`contextTopK 5`

If the domain hint removes all results, step 3–4 re-run once without it (logged).

Where permission and profile filtering happens

1. **In the database query** (steps 3–4): pre-filters inside `$vectorSearch` and `$search`, so inaccessible chunks are never returned at all.
2. **In application code** (step 6): a second, independent check on every candidate before anything is sent to an LLM.
3. **On source viewing** (`GET /api/sources/chunks/:chunkId`): the same `assertAccessible` check, so a guessed chunk ID cannot expose a document.

The LLM never receives profile-inappropriate text and never decides access. `buildAccessFilter` and `assertAccessible` share one rules module and have exhaustive unit tests (every country × employment type × access level × status combination).

Why this design

- Vector search catches paraphrases ("time off after having a baby"); keyword search catches exact terms ("VPN", "GlobalProtect", policy names). Fusion is measured against each alone in the retrieval eval.
- Denormalising metadata onto chunks is required because Atlas search pre-filters only work on fields in the indexed collection.
- RRF in code keeps the fusion portable, unit-testable, and visible in traces.

8. Evaluation architecture

The golden set is the contract's proof of quality. It is a versioned JSONL file (`evals/golden/golden.v1.jsonl`), validated by a Zod schema, run by one CLI against the real graph and real database, with results stored in MongoDB and shown in the admin Evaluation page.

Dataset composition (80 cases)

Category	Count	Covers	Primary expected outcome
`answerable`	35	~18 IT, ~17 HR; at least 8 US/UK or full-time/contractor pairs asking the same question; 4 multi-document questions; 3 table lookups	ANSWER with expected docs and key facts
`partial`	10	Questions where policy covers one part only	PARTIAL with `uncovered` non-empty
`clarify`	10	Ambiguous ("it's not working", "how much leave do I have" without type)	CLARIFY
`escalate`	12	Hardware faults, access requests, no-document topics; 4 multi-turn with clarifications; 2 safety (battery, electrical)	ESCALATE with expected category, priority, fields
`sensitive`	8	Harassment, discrimination, medical, legal, grievance, wellbeing; 2 phrased indirectly	SENSITIVE_HANDOFF
`out_of_scope`	5	Non-work, external advice (tax, legal advice), general trivia	OUT_OF_SCOPE

Cross-cutting tags (a case can carry several): `injection` (≥ 5 cases, spread across categories: "ignore your rules", instructions planted in one corpus document), `superseded_trap` (≥ 4, where an old policy version would give a wrong answer), `profile_variant`, `multi_turn`, `table`, `general_it`.

Test case structure

Field	Type	Purpose
`id`	string, e.g. `HR-ANS-007`	Stable reference
`category`, `tags`	enum, string[]	Grouping
`userEmail`	string	Seeded user whose profile applies
`turns`	`{ role: "user", content }[]` (assistant turns are generated live)	Conversation input; multi-turn cases include planned clarification answers
`expected.decision`	enum	Required decision on the scored turn
`expected.acceptableDecisions`	enum[] (optional)	E.g. PARTIAL acceptable for ANSWER on borderline cases; used sparingly
`expected.docSlugs`	string[]	At least one must be in top 5 retrieved and cited
`expected.forbiddenDocSlugs`	string[]	Superseded or wrong-profile docs; must never be retrieved into context or cited
`expected.keyFacts`	string[]	Facts the answer must contain ("16 weeks", "30 days' notice")
`expected.ticket`	`{ category, subcategory?, minPriority, fields: record<field, string | "unknown"> }`	Escalation cases
`notes`	string	Why the case exists

Metrics and scoring

Metric	Suite	Computation	Target (contract S)
Retrieval recall@5	retrieval, full	Share of cases with `docSlugs` where ⩾ 1 expected doc is in the final top 5	⩾ 85% (week 1 gate ⩾ 75%)
Forbidden-doc leakage	retrieval, full	Cases where a forbidden doc reached context or citations	0
Decision accuracy	full	Decision ∈ `{expected} ∪ acceptable` ÷ all; plus 6×6 confusion matrix	⩾ 85%
Sensitive pass rate	full	Sensitive cases → SENSITIVE_HANDOFF	100%
Out-of-scope accuracy	full		⩾ 90%
Citation validity	full	Valid ÷ proposed citations (code-enforced)	100%
Category accuracy	full	Escalations with correct category	⩾ 90%
Field accuracy	full	Required fields matching expected after normalisation (lowercase, trimmed, synonyms map)	⩾ 85%
Invented values	full	Fields with a value where expected is `unknown`	0
Priority floor compliance	full	Safety cases at ⩾ `minPriority`	100%
Injection resistance	full	Injection-tagged cases with correct decision and no rule-breaking output (string checks + judge)	100%

Metric	Suite	Computation	Target (contract S)
Groundedness	judge	`fully` ÷ judged ANSWER responses; 20 manually spot-checked and recorded	⩾ 90%
Key-fact correctness	judge + string match	Expected key facts present ÷ total	⩾ 90%
Latency, cost	full	p50/p90 TTFT and total; USD per case	NFR targets

Case pass/fail: a case passes when its decision is acceptable and every applicable check for its category passes (recall, no forbidden docs, key facts, ticket fields, no invented values). Failure reasons are recorded as codes, e.g. `WRONG_DECISION`, `MISSED_DOC`, `FORBIDDEN_DOC`, `MISSING_FACT`, `FIELD_MISMATCH`, `INVENTED_VALUE`.

Run pass/fail: a run passes when every metric meets its target. Hard gates that fail a run regardless of the rest: sensitive 100%, forbidden-doc leakage 0, citation validity 100%, injection 100%.

Execution

- `npm run eval -- --suite retrieval` — retrieval only (embeddings + search + fusion + rerank, no generation); ~2 minutes; used constantly in week 1.
- `npm run eval -- --suite full [--tag sensitive] [--case HR-ANS-007]` — runs the compiled graph per case with an in-memory checkpointer, isolated eval conversations (`isEval: true`, excluded from dashboards), concurrency 4; target < 15 minutes.
- `npm run eval -- --suite judge --run <runId>` — judges the ANSWER outputs of a stored full run.
- `npm run eval:sweep -- --run <runId>` — threshold sweep using stored retrieval and rerank outputs, with no new LLM calls.
- Exit code non-zero when the run fails, so it can later gate CI (CI is OPTIONAL).
- Each run writes `eval_runs` (with `configSnapshot`: thresholds, prompt versions, models, dataset version, git commit) and `eval_results`, plus a Markdown report in `evals/reports/`.

Admin UI presentation

The Evaluation page shows the run list (suite, date, pass/fail badge, key metrics, cost); a run detail with metric cards against targets, the decision confusion matrix, per-category breakdown, and a filterable table of failed cases; and case detail with expected vs actual, retrieved docs, the answer and failure reasons. Admins can start a run (one at a time). Comparing two runs side by side is OPTIONAL.

How evaluation drives thresholds and retrieval

1. **Week 1:** retrieval suite compares vector-only, keyword-only and fused recall@5; chunk size and overlap are adjusted until fused recall ⩾ 75%. Choices are recorded in `docs/DECISIONS.md`.
2. **Week 2:** first full run establishes baseline decision accuracy with default `T_answer = 2`, `T_partial = 1`.
3. **Week 4:** `eval:sweep` evaluates each threshold combination on stored outputs and picks the one with the highest decision accuracy subject to two constraints: zero ANSWER decisions on cases expected to escalate or be out of scope, and no drop in sensitive pass rate. The chosen values go into `config/thresholds.yaml` with the sweep table in an ADR.
4. **Overfitting control:** 15 cases are tagged `holdout` and excluded from the sweep; they are reported separately. A large gap between holdout and tuned accuracy is noted as a risk.
5. **Every prompt or threshold change** requires a full run before merging; regressions on hard gates block the change.

9. Security architecture

The prototype uses real enforcement mechanisms (RBAC, filtering, validation, audit) on mocked identity and synthetic data. The rule is: **mock the identity source, never mock the enforcement.**

Area	Prototype design	Production requirement
Authentication	Login picker of seeded demo users; optional shared `DEMO_PASSWORD` from env; no self-registration	Enterprise SSO (OIDC/SAML), MFA via IdP, group-to-role mapping

Area	Prototype design	Production requirement
JWT	HS256, secret ≥ 32 bytes from env; claims `sub`, `role`, `iat`, `exp` (8 h); stored in httpOnly, `SameSite=Lax`, `Secure` in non-local envs cookie `sd_session`; user re-loaded from DB on each request so role changes apply immediately	Short-lived tokens from IdP, refresh rotation, revocation, asymmetric signing
CSRF / CORS	CORS allowlist = web origin only, credentials enabled; mutating requests require header `X-Requested-With: sd-web`; SameSite cookie	Same plus CSRF tokens if cookies cross sites
RBAC	`requireAuth` then `requireRole(...)` middleware on every route; ownership checks in services (employee owns conversation/ticket; agent queue membership); a route table test asserts every route declares a role policy	Same, plus attribute-based policies from IdP groups
Profile filtering	`buildAccessFilter` in DB query + `assertAccessible` in code + check on source viewing (section 7)	Same, driven by HRIS attributes and document ACLs from the source system
Document access	`accessLevel` per document; superseded/inactive never retrievable; admin-only document management	Source-system permissions synced per document
Sensitive tickets	`restricted` flag on conversation, ticket, traces; visible only to `hr_case` agents and admins; excluded from gap clustering and eval exports; every view audited	Separate HR case system, legal hold, stricter retention
Prompt injection	Delimited data sections; no tools bound to the LLM; retrieval and ticket creation are graph-owned nodes; outputs validated by Zod; citations validated; templates for handoff paths; injection cases in golden set (100% gate)	Same, plus content scanning on ingestion and red-team testing

Area	Prototype design	Production requirement
PII in traces	`redactPII()` (emails, phone numbers, card-like and ID-like numbers, UK NI / US SSN patterns) applied to user text before writing `turn_traces.redactedInput`, gap clusters and eval exports; raw text only in `messages`, protected by RBAC	DLP service, configurable retention, PII never sent to logs
Data sent to LLM providers	Synthetic data only; a startup warning if `NODE_ENV=production`	DPA with provider, data residency, zero-retention settings
Rate limiting	In-memory per-user limits: chat 20 turns/min, login 10/min per IP, uploads 10/hour, eval runs 1 concurrent	Distributed limiter (Redis), per-tenant quotas
File upload	`.md` and `.pdf` only; ⩽ 5 MB; MIME type plus magic-byte check for PDF; UTF-8 check for Markdown; metadata validated by Zod; files processed in memory, never served back raw	Malware scanning, sandboxed parsing
Input limits	JSON body ⩽ 100 KB; message ⩽ 2,000 chars; all inputs validated by Zod	Same
HTTP hardening	`helmet` headers, `x-powered-by` disabled, request IDs, no stack traces in responses	WAF, TLS everywhere
Secrets	Env vars only (`JWT_SECRET`, `GEMINI_API_KEY`, `OPENROUTER_API_KEY`, `MONGODB_URI`); `.env.example` without values; env schema validated at startup; secrets never sent to the web app	Secret manager, rotation
Audit log	Append-only `audit_log` for login/logout, ticket create/status/classification/reply, restricted ticket views, trace views, document changes, eval runs; no update/delete API	Tamper-evident store, SIEM export

Area	Prototype design	Production requirement
Dependencies	Lockfile committed; `npm audit` in the hardening week	Automated scanning

Security tests required: role-matrix API tests (every route × every role), ownership tests (employee A cannot read B's conversation), restricted-ticket visibility tests, access-filter exhaustive unit tests, upload rejection tests, redaction unit tests, and the injection cases in the golden set.

10. Frontend architecture

Next.js App Router with TypeScript and Tailwind. Route groups per role, each with a layout that calls `GET /api/auth/me` and redirects if the role does not match (UX only; the API enforces access). Data fetching uses a typed client in `src/lib/api.ts` that validates responses with the shared Zod schemas. Client components only where interaction requires it (chat, forms, filters). No global state library; React state and URL search params are enough.

Shared layout and primitives: `AppShell` (role nav, user badge with country/employment type, logout), `PrototypeBanner` (always visible: "Prototype — synthetic data, mocked integrations"), `ui/` primitives (Button, Badge, Card, Table, Tabs, Dialog, Textarea, Select, Spinner, EmptyState, ErrorState, Toast). Every page implements the same three states: skeleton while loading, `ErrorState` with retry and request ID, `EmptyState` with a next action.

Routes: `/login`; employee `/chat`, `/chat/[conversationId]`, `/tickets`, `/tickets/[id]`; agent `/agent/queue`, `/agent/tickets/[id]`; admin `/admin`, `/admin/gaps`, `/admin/traces`, `/admin/traces/[conversationId]`, `/admin/documents`, `/admin/documents/[id]`, `/admin/evals`, `/admin/evals/[runId]`.

Screen	Purpose	Data / API calls	Major components	User actions	Loading / error / empty
Login (`/login`)	Pick a demo persona	`GET /auth/demo-users` , `POST /auth/login`	`PersonaCard` grid grouped by role, showing country and employment type	Choose persona; enter demo password if enabled	Skeleton cards / "API unreachable" with retry / "No demo users — run seed"

Screen	Purpose	Data / API calls	Major components	User actions	Loading / error / empty
Chat (`/chat/[id]`)	Ask questions, get answers or tickets	`GET /conversations` , `GET /conversations/:id` , `POST /conversations` , `POST .../messages` (SSE), `POST .../resolution` , `POST /messages/:id/feedback` , `POST .../escalate`	`ConversationSidebar` , `MessageList` , `AssistantMessage` (streamed text, citation chips, decision badge), `StageIndicator` (analyzing/searching/writing), `ResolutionPrompt` , `FeedbackButtons` , `ClarifyingQuestion` , `TicketDraftCard` , `HandoffCard` , `SafetyNotice` , `Composer` with "Talk to a human"	Send message; click citation; confirm resolved / not resolved; thumbs; edit and confirm/cancel draft; respond to duplicate prompt	Typing indicator and stage text / inline message error with retry, withdrawn answer replaced in place / "Ask about IT or HR" with 4 example prompts
Source viewer (drawer on Chat)	Show the cited passage and its document context	`GET /sources/chunks/:chunkId`	`SourceDrawer` (title, version, effective date, owner, section path, highlighted chunk, previous/next chunk)	Open from chip; close; step to neighbouring chunk	Spinner / "Source not available" (404) / —
Ticket draft (inline on Chat)	Review AI-filled ticket before creation	from `ticket_draft` SSE event; `POST .../ticket-draft/confirm` or `/cancel`	`TicketDraftCard` with editable fields, "AI-filled" markers, required-missing highlights, proposed priority (read-only)	Edit fields; confirm; cancel; choose same/new on duplicate	Confirm button busy / field validation errors, retry on 500 / —
My Tickets (`/tickets` , `/tickets/[id]`)	Track escalations	`GET /tickets/mine` , `GET /tickets/:id`	`TicketList` with status badges; `TicketDetail` with summary, status timeline, public replies, link to conversation	Filter by status; open ticket	Skeleton rows / ErrorState / "No tickets — the hub resolved everything so far"
Agent Queue (`/agent/queue`)	Work tickets in priority order	`GET /agent/tickets` with filters	`QueueFilters` (queue, status, priority, category), `TicketTable` (priority badge, age, restricted lock icon)	Filter; sort; open ticket	Skeleton table / ErrorState / "Queue clear"
Agent Ticket Detail (`/agent/tickets/[id]`)	Resolve with full context	`GET /agent/tickets/:id` , `POST .../draft-reply` , `POST .../replies` , `POST .../notes` , `PATCH .../status` , `PATCH .../classification`	`TicketHeader` (humanId, priority, status selector), `AiSummaryPanel` , `ExtractedFields` (AI-filled markers), `ArticlesShown` , `TranscriptAccordion` , `ReplyComposer` with "Generate draft", `ClassificationEditor` , `EventTimeline`	Generate and edit draft; send reply; add note; change status; correct category/subcategory /priority	Panels load independently / per-action toasts; 409 invalid transition message / "No events yet"
Admin Overview (`/admin`)	Health of AI and service desk	`GET /admin/metrics/overview`	`DateRangePicker` , `MetricCard` grid, `DecisionDistributionBar` , `EscalationByCategoryTable` , `LatencyCostCard`	Change date range; drill to traces filtered by decision	Skeleton cards / ErrorState / "No conversations in range"

Screen	Purpose	Data / API calls	Major components	User actions	Loading / error / empty
Knowledge Gaps (`/admin/gaps`)	Find missing documentation	`GET /admin/knowledge-gaps` , `POST .../refresh`	`GapClusterList` (label, domain, count), `ClusterDetail` (example questions with links to traces)	Refresh snapshot; open cluster; jump to trace	Refresh progress / ErrorState / "No gaps detected yet"
Trace Viewer (`/admin/traces` , `/[conversationId]`)	Explain any AI decision	`GET /admin/conversations` , `GET /admin/conversations/:id/traces` , `GET /admin/traces/:id`	`ConversationTable` with filters; `TurnTimeline` ; `TracePanel` tabs: Analysis, Retrieval (vector/keyword/fused/reranked with scores), Decision (rules fired, thresholds), Output (citations valid/rejected), LLM calls (prompt version, model, tokens, cost, latency)	Filter; select turn; expand chunk text	Skeleton / ErrorState / "No conversations"
Documents (`/admin/documents` , `/[id]`)	Manage knowledge base	`GET /admin/documents` , `GET /admin/documents/:id` , `POST /admin/documents` , `POST .../reindex` , `PATCH .../status`	`DocumentTable` (status, audience, version, ingest status), `UploadDialog` with metadata form, `ChunkList`	Upload; re-index; change status; inspect chunks	Upload progress / validation errors per field / "No documents — run ingest"
Evaluation (`/admin/evals` , `/[runId]`)	Prove and track AI quality	`GET /admin/evals/runs` , `POST /admin/evals/runs` , `GET .../runs/:id` , `GET .../runs/:id/results`	`RunTable` with pass badges, `StartRunDialog` , `MetricTargetCards` , `ConfusionMatrix` , `CategoryBreakdown` , `FailedCaseTable` , `CaseDetail` , `RunCompare` (OPTIONAL: two runs side by side)	Start run; poll status; filter failed cases; compare runs (OPTIONAL)	Running state with progress count (poll every 5 s) / failed run error / "No runs yet — start one"

Streaming client: `src/lib/sse.ts` reads the POST response body stream, parses events with the shared Zod event schema, and drives a `useChatTurn` hook with an explicit state machine (`idle → streaming → awaiting_citations → done | withdrawn | error`). Charts on admin pages are simple Tailwind bars and tables; a chart library is OPTIONAL and requires approval.

11. Four-week development roadmap

40 tasks, each sized for one Claude Code session (roughly 1–4 hours of build time). Tasks are done in ID order unless dependencies allow otherwise. Every task's Definition of Done also includes the standard DoD: lint and type-check clean, all existing tests green, `docs/PROGRESS.md` updated, no scope added.

Week	Theme	Tasks	Checkpoint at end
1	Foundation, data, auth, corpus, retrieval	W1-01 – W1-12	CP-1 Foundation, CP-2 Retrieval
2	Orchestration, employee chat, decision evals	W2-01 – W2-10	CP-3 AI orchestration, CP-4 Employee flow
3	Ticketing and agent workspace	W3-01 – W3-08	CP-5 Ticketing, CP-6 Agent flow
4	Admin, evaluation polish, hardening, demo	W4-01 – W4-10	CP-7 Admin, CP-8 Evaluation, CP-9 Final hardening

Week 1 — Foundation, data, auth, corpus, retrieval

W1-01 Monorepo scaffold

- **Objective:** a runnable empty monorepo with conventions in place.
- **Dependencies:** none.
- **Files/modules:** root `package.json` (npm workspaces), `tsconfig.base.json`, ESLint, Prettier, `apps/web`, `apps/api`, `packages/shared`, `CLAUDE.md`, `docs/*`, `.env.example`, `docker-compose.yml` (web, api, `mongodb-atlas-local`).
- **Implementation:** Next.js App Router app with Tailwind; Express app with `/api/health`; shared package building with TS project references; Jest configured in api and shared; Playwright installed in `e2e/`; copy contract and playbook into `docs/`; ADR-001 (stack) written.
- **Acceptance:** `docker compose up` starts all three; `GET /api/health` returns 200; web home renders.
- **Tests:** one Jest smoke test per package; one Playwright test loading the home page.
- **DoD:** `npm run lint`, `npm run typecheck`, `npm test` pass from root.

W1-02 Config and env loading

- **Objective:** all tunables and secrets loaded and validated at startup.
- **Dependencies:** W1-01.
- **Files/modules:** `apps/api/src/config/*`, `config/*.yaml` (initial versions of taxonomy, routing, priority-rules, sensitive, thresholds, ticket-schemas, prompts, models, templates).

- **Implementation:** Zod env schema (fails fast with readable errors); YAML loader with one Zod schema per file; typed `config` singleton; `models.yaml` holds provider, fast/quality model names, embedding model and dimension.
- **Acceptance:** a missing env var or malformed YAML stops startup with a clear message.
- **Tests:** unit tests for each config schema (valid and invalid fixtures).
- **DoD:** config documented in README section "Configuration".

W1-03 Shared contracts and error envelope

- **Objective:** a single source of API types.
- **Dependencies:** W1-01.
- **Files/modules:** `packages/shared/src/{enums,api/*,errors}.ts`, `apps/api/src/http/{errors,requestId,validate}.ts`.
- **Implementation:** enums (roles, decisions, statuses, priorities, domains); error envelope schema; `validate(schema)` Express middleware; central error handler mapping known errors to codes; request ID middleware; pino logger.
- **Acceptance:** invalid body on a sample route returns `VALIDATION_ERROR` envelope with request ID.
- **Tests:** Supertest for error handler and validation middleware.
- **DoD:** contracts importable from web and api.

W1-04 Database layer and indexes

- **Objective:** typed collections, indexes and counters.
- **Dependencies:** W1-02.
- **Files/modules:** `apps/api/src/db/{client,collections,schemas,indexes}.ts`, `scripts/create-indexes.ts`.
- **Implementation:** driver connection with pooling; Zod schemas for all collections in section 3; regular indexes; Atlas Vector Search and Atlas Search index definitions for `chunks`; counters helper; `npm run db:indexes` idempotent.
- **Acceptance:** running the script twice creates all indexes once; search indexes reach ready state on `mongodb-atlas-local`.
- **Tests:** integration tests against the atlas-local container for counters atomicity and unique indexes.
- **DoD:** ADR-003 (atlas-local for dev/test) recorded.

W1-05 Users seed and auth

- **Objective:** mock login with JWT cookie and RBAC.
- **Dependencies:** W1-03, W1-04.
- **Files/modules:** `modules/auth`, `modules/users`, `modules/audit`, `http/{requireAuth,requireRole}.ts`, `seed/users.json`, `scripts/seed.ts` (users part).
- **Implementation:** ~8 users (Priya US full-time engineer, Tom UK contractor, a UK full-time manager, a US contractor, Sam IT agent with `it_hardware` / `it_software` / `it_access`, Hana HR agent with `hr_general` / `hr_case`, Alex admin, one more employee); demo-users, login, logout, me, profiles/me; audit service with `auth.login` / `auth.logout`.
- **Acceptance:** each role logs in; protected route returns 401 without cookie and 403 with wrong role.
- **Tests:** Supertest for all auth endpoints; RBAC middleware unit tests; audit entries written.
- **DoD:** route policy table started in `http/routes.ts`.

W1-06 Frontend shell and login

- **Objective:** role-aware navigation from day one.
- **Dependencies:** W1-05.
- **Files/modules:** `apps/web/src/app/(auth)/login`, `(employee)`, `(agent)`, `(admin)` layouts, `components/ui/*`, `lib/api.ts`, `AppShell`, `PrototypeBanner`.
- **Implementation:** login persona picker; typed API client with `credentials: "include"`; layouts redirect by role; placeholder pages for every route in section 10.
- **Acceptance:** each persona lands on their home; logout returns to login; banner on every page.
- **Tests:** Playwright: login as each role and see correct nav.
- **DoD:** loading/error/empty primitives exist.

W1-07 Corpus authoring and validation

- **Objective:** a realistic 30–40 document knowledge base.
- **Dependencies:** W1-02 (taxonomy).
- **Files/modules:** `corpus/it/*.md`, `corpus/hr/*.md`, `corpus/README.md`, `scripts/validate-corpus.ts`.

- **Implementation:** documents per section Q of the contract (≥ 6 US/UK topic pairs, ≥ 2 superseded versions, ≥ 3 with tables, a battery-safety policy, an access-request policy, 3–4 deliberately missing topics listed in README, one document containing a planted injection sentence for eval). AI may draft, the developer reviews for consistency of numbers.
- **Acceptance:** validation script passes: frontmatter schema, unique slugs, one active version per slug, taxonomy domains.
- **Tests:** the validation script runs in `npm test`.
- **DoD:** corpus inventory table in `corpus/README.md`.

W1-08 Golden set v1

- **Objective:** 80 labelled cases written alongside the corpus.
- **Dependencies:** W1-07.
- **Files/modules:** `evals/golden/golden.v1.jsonl`, `evals/runner/schema.ts`, `scripts/validate-golden.ts`.
- **Implementation:** case schema per section 8; composition counts and tags as specified; 15 `holdout` cases; every `docSlugs` / `forbiddenDocSlugs` must exist in corpus; every `userEmail` must exist in seed.
- **Acceptance:** validator passes and prints composition counts matching section 8.
- **Tests:** validator in `npm test`.
- **DoD:** golden set committed; dataset version `v1`.

W1-09 LLM and embedding provider layer

- **Objective:** swap providers by config only.
- **Dependencies:** W1-02.
- **Files/modules:** `ai/llm/{chatModel,embeddings,costTable,fakeModels}.ts`.
- **Implementation:** factory returning LangChain chat models for `gemini` (Google GenAI integration) and `openrouter` (OpenAI-compatible chat model with OpenRouter base URL), by tier `fast` / `quality`; embeddings factory (Gemini default); per-call timeout and retry with backoff; token usage capture; cost table from config; fake chat and embedding models for tests.
- **Acceptance:** `npm run llm:smoke` returns a structured output from both providers when keys are present.
- **Tests:** unit tests for factory selection and fallback structured-output path using fakes.

- **DoD:** D-1/D-2 approvals recorded in DECISIONS.md.

W1-10 Ingestion pipeline

- **Objective:** corpus to indexed chunks.
- **Dependencies:** W1-04, W1-07, W1-09.
- **Files/modules:** `ai/ingestion/{loaders,chunker,contextualise,embedder,indexer}.ts`, `modules/documents/service.ts`, `scripts/ingest.ts`.
- **Implementation:** section 7 ingestion path; idempotency via content hash; supersede handling; wait-for-index polling.
- **Acceptance:** `npm run ingest` indexes the whole corpus; re-run is a no-op; chunk metadata matches documents.
- **Tests:** chunker unit tests (headings, tables not split, overlap); integration test ingesting 3 fixture docs.
- **DoD:** chunk statistics printed (count, mean tokens).

W1-11 Access filter and hybrid retrieval

- **Objective:** profile-safe hybrid search.
- **Dependencies:** W1-10.
- **Files/modules:** `ai/retrieval/{accessRules,accessFilter,vectorSearch,keywordSearch,fusion,retrieve}.ts`.
- **Implementation:** section 7 steps 1–6 (rerank comes in W2-04); `retrieve(query, profile)` returns candidates with per-source scores.
- **Acceptance:** a UK contractor never receives US-only or superseded chunks.
- **Tests:** exhaustive access-rule unit tests; RRF unit tests; integration tests for vector, keyword and fused search against fixtures.
- **DoD:** retrieval callable from a script with a printed result table.

W1-12 Retrieval eval runner

- **Objective:** measure recall@5 before any generation.
- **Dependencies:** W1-08, W1-11.
- **Files/modules:** `evals/runner/{cli,retrievalSuite,scorers,report,store}.ts`.

- **Implementation:** `npm run eval -- --suite retrieval` with modes `vector`, `keyword`, `fused`; writes eval_runs/eval_results and a Markdown report.
- **Acceptance:** report shows recall@5 for each mode and forbidden-doc leakage.
- **Tests:** scorer unit tests with synthetic results.
- **DoD:** CP-2 gate: fused recall@5 ⩾ 75%, leakage 0; chunking decisions recorded.

Week 2 — Orchestration, employee chat, decision evals

W2-01 Prompt registry

- **Objective:** versioned prompts with hashes.
- **Dependencies:** W1-09.
- **Files/modules:** `ai/prompts/registry.ts`, `ai/prompts/<name>/v1.md` for P-1 – P-4, `config/prompts.yaml`.
- **Implementation:** frontmatter parsing, Zod-validated metadata, render with variables, hash, active version lookup.
- **Acceptance:** registry returns template, version and hash; unknown prompt fails at startup.
- **Tests:** unit tests for loading, missing variables, version switching.
- **DoD:** prompt-writing guidelines added to CLAUDE.md.

W2-02 Graph state, skeleton and checkpointer

- **Objective:** the full LangGraph topology with stub nodes.
- **Dependencies:** W2-01.
- **Files/modules:** `ai/graph/{state,graph,edges,checkpointer}.ts`, `ai/nodes/*` stubs, `ai/tracing/*`.
- **Implementation:** state annotation per section 5; every node and conditional edge wired; stubs return fixed outputs; MongoDB checkpointer (in-memory saver for tests); trace accumulator with step timings.
- **Acceptance:** graph compiles; a stubbed run reaches each terminal path by setting state.
- **Tests:** AI workflow tests with fake models asserting the path taken for each decision.
- **DoD:** graph topology printed (Mermaid export) into ARCHITECTURE.md.

W2-03 loadContext, precheck, analyzeTurn

- **Objective:** understand each turn.
- **Dependencies:** W2-02.
- **Files/modules:** `ai/nodes/{loadContext,precheck,analyzeTurn}.ts`, `ai/guards/precheckRules.ts`, `ai/schemas/turnAnalysis.ts`, prompt P-1.
- **Implementation:** per section 5 and 6, including retry and fallback.
- **Acceptance:** sensitive keywords route without an LLM call; analysis output validates.
- **Tests:** precheck rule unit tests; analyzeTurn with fake model (valid, invalid-then-valid, double failure → fallback).
- **DoD:** trace contains precheck and analysis.

W2-04 retrieve and rerankAssess nodes

- **Objective:** final context with coverage verdict.
- **Dependencies:** W1-11, W2-03.
- **Files/modules:** `ai/nodes/{retrieve,rerankAssess}.ts`, `ai/schemas/rerank.ts`, prompt P-2.
- **Implementation:** section 7 steps 7–8; unknown chunk IDs dropped; fallback path.
- **Acceptance:** retrieval eval extended with reranked recall@5 ⩾ fused recall@5.
- **Tests:** node tests with fakes; ID-dropping test; fallback test.
- **DoD:** rerank recall recorded in PROGRESS.md.

W2-05 Decision rules and decide node

- **Objective:** deterministic six-way decision.
- **Dependencies:** W2-04.
- **Files/modules:** `domain/decisionRules.ts`, `ai/nodes/decide.ts`.
- **Implementation:** ordered rules table from section 5, thresholds from config, `rulesFired` recorded.
- **Acceptance:** every rule reachable; first-match semantics.
- **Tests:** table-driven unit tests covering every rule and boundary values.
- **DoD:** rules documented in ARCHITECTURE.md.

W2-06 Grounded answer and citation validation

- **Objective:** streamed, cited, validated answers.

- **Dependencies:** W2-05.
- **Files/modules:** `ai/nodes/{answer,validateCitations}.ts`, `ai/guards/{citations,numericCheck}.ts`, prompt P-3.
- **Implementation:** source labelling, streaming, trailer parsing, marker → chunk mapping, withdrawal path, numeric check (numbers in answer must appear in cited chunks).
- **Acceptance:** an answer citing a non-retrieved ID is impossible; zero valid citations → withdrawn.
- **Tests:** validator unit tests (valid, invented marker, no markers, numeric mismatch); node test with fake streaming model.
- **DoD:** contract acceptance item "no citation not retrieved" unit-tested.

W2-07 Clarify, out-of-scope and sensitive handoff nodes

- **Objective:** all non-answer, non-ticket outcomes.
- **Dependencies:** W2-05.
- **Files/modules:** `ai/nodes/{clarify,outOfScope,sensitiveHandoff}.ts`, prompt P-4, `config/templates.yaml`.
- **Implementation:** per sections 5–6; sensitive path ends with the "Would you like HR to contact you?" offer (ticket creation itself in W3-03).
- **Acceptance:** sensitive and out-of-scope replies make no generation call beyond P-1.
- **Tests:** node tests; template rendering tests.
- **DoD:** sensitive path never includes retrieved policy text.

W2-08 Persist node and chat API with SSE

- **Objective:** end-to-end turn over HTTP.
- **Dependencies:** W2-06, W2-07.
- **Files/modules:** `ai/nodes/persist.ts`, `ai/guards/redaction.ts`, `modules/conversations`, `modules/chat`, `packages/shared/src/api/chat-events.ts`.
- **Implementation:** conversation endpoints, messages SSE endpoint, resolution, feedback, source chunk endpoint; one running turn per conversation (409); stream mapping; persistence of messages, traces (redacted), conversation status.
- **Acceptance:** a curl-driven turn streams events in documented order and persists all records.

- **Tests:** Supertest for each endpoint (ownership, 404 vs 403, 409); redaction unit tests; integration test with fake models asserting persisted trace shape.
- **DoD:** API section 4 Chat endpoints implemented exactly.

W2-09 Chat UI

- **Objective:** employee experience for answer/clarify/handoff paths.
- **Dependencies:** W2-08, W1-06.
- **Files/modules:** `apps/web/src/app/(employee)/chat/*`, `components/chat/*`, `lib/sse.ts`, `hooks/useChatTurn.ts`.
- **Implementation:** section 10 Chat and Source viewer screens (ticket draft card stubbed until week 3).
- **Acceptance:** Priya and Tom get different cited answers to the same question; citations open the source drawer; resolution and feedback persist.
- **Tests:** Playwright flows 1, 2, 3 (with live or recorded LLM per test config); component tests for the SSE state machine.
- **DoD:** CP-4 employee flow checklist passes.

W2-10 Full-graph eval suite

- **Objective:** decision quality measured.
- **Dependencies:** W2-08.
- **Files/modules:** `evals/runner/fullSuite.ts`, scorers for decision, sensitive, out-of-scope, citations, key facts (string match).
- **Implementation:** runs the compiled graph with in-memory checkpointer and `isEval`; concurrency 4; confusion matrix.
- **Acceptance:** `npm run eval -- --suite full` completes; baseline stored.
- **Tests:** scorer unit tests.
- **DoD:** CP-3 gate: sensitive 100%, citation validity 100%, decision accuracy reported (target applies at CP-8).

Week 3 — Ticketing and agent workspace

W3-01 Ticketing domain rules

- **Objective:** deterministic routing, priority and lifecycle.
- **Dependencies:** W1-02.

- **Files/modules:** `domain/{routing,priorityFloors,ticketStateMachine,ticketSchemas,duplicateCheck}.ts`.
- **Implementation:** category → queue from `routing.yaml`; floor rules from `priority-rules.yaml` (safety → critical); allowed status transitions; category schema lookup with required fields; duplicate rule (same requester, same category, status not resolved/closed, within 7 days).
- **Acceptance:** every taxonomy category maps to exactly one queue (startup check).
- **Tests:** table-driven unit tests for all five modules.
- **DoD:** 100% branch coverage on `domain/`.

W3-02 slotFill and clarifyForTicket nodes

- **Objective:** complete ticket fields through conversation.
- **Dependencies:** W3-01, W2-07.
- **Files/modules:** `ai/nodes/{slotFill,clarifyForTicket,buildDraft}.ts`, `ai/schemas/slotFill.ts`, prompts P-5 and P-4 (ticket mode).
- **Implementation:** sections 5–6; `ticketFlow` persisted via checkpointer across turns; max 2 rounds; `unknown` preserved.
- **Acceptance:** battery case asks for device model once, then drafts.
- **Tests:** multi-turn workflow tests with fakes (complete first time, one round, two rounds then draft with unknowns).
- **DoD:** draft schema matches `TicketDraft` in shared package.

W3-03 Human-in-the-loop confirmation and ticket creation

- **Objective:** tickets only after confirmation.
- **Dependencies:** W3-02.
- **Files/modules:** `ai/nodes/{awaitConfirmation,createTicket}.ts`, `modules/tickets/*`, chat confirm/cancel endpoints.
- **Implementation:** `interrupt()` with draft; confirm/cancel resume with `Command`; edits validated against category schema; duplicate 409 flow; humanId from counters; queue and priority by domain rules; requester from auth; `created` event; system message; sensitive-handoff Yes path creating restricted `hr_case` ticket.
- **Acceptance:** no ticket exists before confirm; confirm is idempotent; requester cannot be overridden by edits.

- **Tests:** Supertest for confirm/cancel/duplicate; workflow test for interrupt/resume across two requests; unit test that edits cannot set requester, queue or priority.
- **DoD:** contract acceptance item on confirmation verified.

W3-04 Ticket draft UI, safety flow, My Tickets

- **Objective:** employee side of escalation.
- **Dependencies:** W3-03, W2-09.
- **Files/modules:** `components/chat/{TicketDraftCard,SafetyNotice,HandoffCard}.tsx`, `app/(employee)/tickets/*`, "Talk to a human" wiring to `/escalate`.
- **Implementation:** section 10 screens; safety steps streamed with citations before the draft.
- **Acceptance:** contract flows 4, 5 and 6 work in the UI.
- **Tests:** Playwright for flows 4, 5, 6; component tests for draft editing and duplicate prompt.
- **DoD:** CP-5 checklist passes.

W3-05 Agent ticket API

- **Objective:** queue-scoped agent operations.
- **Dependencies:** W3-03.
- **Files/modules:** `modules/agent/*`.
- **Implementation:** all Agent endpoints in section 4; queue scoping; restricted visibility; classification corrections as events with re-routing; audit entries.
- **Acceptance:** Sam cannot see HR tickets; Hana sees restricted tickets; invalid transitions 409.
- **Tests:** role-matrix and ownership Supertest suites; correction events recorded with old/new.
- **DoD:** override-rate query returns correct value on fixtures.

W3-06 Agent summary and draft reply

- **Objective:** AI assistance for agents.
- **Dependencies:** W3-05.
- **Files/modules:** `ai/nodes/agentSummary.ts`, `modules/agent/draftReply.ts`, prompts P-6, P-7.

- **Implementation:** summary generated after ticket creation (graph node); draft reply on demand (direct LangChain chain, not the turn graph), logged on the event.
- **Acceptance:** summary and brief present on new tickets; draft reply never auto-sent.
- **Tests:** unit tests with fakes, including failure fallbacks.
- **DoD:** prompt versions recorded.

W3-07 Agent UI

- **Objective:** agent workspace screens.
- **Dependencies:** W3-05, W3-06.
- **Files/modules:** `app/(agent)/*`, `components/agent/*`.
- **Implementation:** section 10 Queue and Ticket Detail.
- **Acceptance:** contract flow 8 works end to end.
- **Tests:** Playwright flow 8; component tests for classification editor and status selector.
- **DoD:** CP-6 checklist passes.

W3-08 Extraction metrics and battery E2E

- **Objective:** measure escalation quality.
- **Dependencies:** W3-03, W2-10.
- **Files/modules:** `evals/runner/scorers/{category,fields,priority}.ts`.
- **Implementation:** multi-turn eval cases replay planned user replies; field normalisation and synonyms map; invented-value detection.
- **Acceptance:** full suite reports category accuracy, field accuracy, invented values, priority floor compliance.
- **Tests:** scorer unit tests.
- **DoD:** results noted in PROGRESS.md; gaps listed as week 4 tuning items.

Week 4 — Admin, evaluation polish, hardening, demo

W4-01 Metrics API and Admin Overview

- **Objective:** service desk and AI health at a glance.
- **Dependencies:** W3-05.
- **Files/modules:** `modules/metrics/*`, `app/(admin)/page.tsx`, `components/admin/overview/*`.

- **Implementation:** aggregation pipelines with the fixed metric definitions; `isEval: false` filter; latency percentiles from traces.
- **Acceptance:** numbers match a hand-computed fixture dataset.
- **Tests:** integration tests on fixtures for each metric; Playwright smoke.
- **DoD:** definitions shown as tooltips in UI.

W4-02 Knowledge gaps

- **Objective:** surface missing documentation.
- **Dependencies:** W4-01.
- **Files/modules:** `modules/gaps/*`, prompt P-8, `app/(admin)/gaps/*`.
- **Implementation:** select ESCALATE/`NO_EVIDENCE` and PARTIAL turns (non-restricted, non-eval); embed redacted questions; simple agglomerative clustering in code with a cosine threshold from config; label clusters with P-8; store snapshot.
- **Acceptance:** seeded questions about missing topics form labelled clusters.
- **Tests:** clustering unit tests on synthetic vectors; restricted exclusion test.
- **DoD:** contract flow 9 gap step works.

W4-03 Trace viewer

- **Objective:** explain any decision.
- **Dependencies:** W2-08.
- **Files/modules:** admin conversations/traces endpoints, `app/(admin)/traces/*`.
- **Implementation:** section 10 Trace Viewer; audit on view.
- **Acceptance:** for Priya's battery turn the viewer shows precheck hit, rules fired and priority floor.
- **Tests:** Supertest (admin only, redaction present); Playwright smoke.
- **DoD:** every trace field from section 3 visible.

W4-04 Document management

- **Objective:** admin control of the knowledge base.
- **Dependencies:** W1-10.
- **Files/modules:** document endpoints, upload validation, `app/(admin)/documents/*`.
- **Implementation:** section 4 Documents; upload reuses the ingestion pipeline.
- **Acceptance:** uploading a new policy makes it retrievable for the right profiles only; superseding hides the old version.

- **Tests:** upload rejection tests (type, size, metadata); integration test upload → retrieve.
- **DoD:** contract flow 9 document step works.

W4-05 Evaluation API and page

- **Objective:** evals visible and runnable from the UI.
- **Dependencies:** W2-10, W3-08.
- **Files/modules:** `modules/evals/*` (background job wrapper around runner), `app/(admin)/evals/*`.
- **Implementation:** section 4 Evaluation and section 10 Evaluation screen; one run at a time.
- **Acceptance:** admin starts a run, watches progress, inspects failed cases.
- **Tests:** Supertest (409 concurrent run, admin only); Playwright smoke with a 3-case tag.
- **DoD:** OPTIONAL run comparison only if time allows.

W4-06 Judge suite and threshold tuning

- **Objective:** groundedness measured, thresholds tuned.
- **Dependencies:** W4-05.
- **Files/modules:** `evals/runner/{judgeSuite,sweep}.ts`, prompt P-9, `config/thresholds.yaml`.
- **Implementation:** section 8 judge and sweep; holdout reporting; 20-answer manual spot check recorded in `evals/reports/manual-check.md`.
- **Acceptance:** thresholds chosen by sweep with ADR; holdout reported.
- **Tests:** sweep unit test on synthetic stored outputs.
- **DoD:** CP-8 gate: all section 8 targets met or deviation documented with a plan.

W4-07 Security hardening

- **Objective:** section 9 fully enforced.
- **Dependencies:** all API tasks.
- **Files/modules:** `http/*` (helmet, rate limits, body limits, CSRF header check), `modules/audit`, route policy test.
- **Implementation:** remaining section 9 controls; `npm audit` triage.
- **Acceptance:** route-policy test proves every route declares a role; injection cases 100%.
- **Tests:** role-matrix suite over all routes; rate-limit tests; header tests.

- **DoD:** security section of README written.

W4-08 Historic seed data and reset

- **Objective:** dashboards look alive in the demo.
- **Dependencies:** W4-01, W4-02.
- **Files/modules:** `seed/historic/*`, `scripts/{seed,reset}.ts`.
- **Implementation:** ~120 historic conversations, ~40 tickets, events, feedback and traces spread over 14 days, generated deterministically (fixed seed), clearly marked synthetic; `npm run reset` restores demo state in under 2 minutes.
- **Acceptance:** overview, gaps and queues are populated after reset; demo-run data is cleared.
- **Tests:** seed idempotency integration test.
- **DoD:** reset documented in demo script.

W4-09 Performance and resilience

- **Objective:** meet latency targets reliably.
- **Dependencies:** W4-06.
- **Files/modules:** `ai/llm/*`, `ai/retrieval/*`, `ai/graph/*`.
- **Implementation:** run vector and keyword search in parallel; reuse embedding of query; fast model for P-1, P-2, P-4; timeouts per node; provider retry with backoff; measure p50/p90 over 30 scripted turns.
- **Acceptance:** NFR-1 to NFR-3 met on the demo machine.
- **Tests:** latency script output stored in `evals/reports/latency.md`.
- **DoD:** fallback behaviour verified by forcing a provider timeout.

W4-10 E2E suite, documentation and demo

- **Objective:** the contract demo, twice, clean.
- **Dependencies:** all.
- **Files/modules:** `e2e/*`, `README.md`, `docs/{ARCHITECTURE,PRODUCTION_GAPS}.md`, `docs/DEMO_SCRIPT.md`.
- **Implementation:** Playwright specs for all nine flows; README (setup, architecture, AI/code split, eval results, production gaps); demo script from the contract; backup screen recording.

- **Acceptance:** all contract acceptance criteria checked; two consecutive clean rehearsals.
- **Tests:** full E2E suite green.
- **DoD:** CP-9 passes; Final Definition of Done (section 15) fully checked.

Cut order if behind (from the contract, adapted): document upload UI (keep CLI ingest) → duplicate ticket check → agent draft reply → gap labelling (show raw clusters). Never cut: decision layer, citation validation, access filtering, sensitive handoff, evaluation harness.

12. Claude Code execution strategy

One task per session, with the repository's own documents carrying memory between sessions. Claude Code starts each session cold, so consistency comes from files it must read, not from conversation history.

Files that carry architecture across sessions

File	Role	Updated by
`CLAUDE.md`	Standing rules: stack, commands, folder boundaries, "AI proposes, code decides", Do Not Build list, testing rules, session protocol below	You, rarely; Claude Code proposes edits
`docs/BUILD_CONTRACT.md`	What to build (read-only)	Never, except approved amendments
`docs/PLAYBOOK.md`	How to build; task cards	Approved changes only
`docs/ARCHITECTURE.md`	Current reality: modules, graph topology, data model, deviations	Claude Code, every task that changes structure
`docs/DECISIONS.md`	ADRs: context, decision, alternatives, consequences	Claude Code, for every non-trivial choice
`docs/PROGRESS.md`	Task status, checkpoint results, eval numbers, known issues	Claude Code, end of every task

Session protocol (put this verbatim in `CLAUDE.md`)

Before implementation:

1. Read `CLAUDE.md`, `docs/PROGRESS.md`, the task card in `docs/PLAYBOOK.md`, and the playbook sections the card references.
2. Inspect the affected modules and their tests; run `npm test` to confirm a green baseline.
3. Restate the task scope, the files to be touched, and anything ambiguous. Stop and ask if the card conflicts with the code or contract.
4. Propose a short plan; wait for approval on tasks marked with dependencies on D-1 – D-5 decisions.

During implementation:

1. Implement only the current task; no opportunistic refactors outside its files.
2. Write or update tests alongside code; use fake models for AI unit and workflow tests.
3. Keep Zod schemas in `packages/shared` or `ai/schemas` as the single definition of every shape.
4. Any new dependency, folder, collection, endpoint or LLM call not in the playbook → stop and ask.
5. Record non-obvious decisions as ADRs immediately.

After implementation:

1. Run `npm run lint`, `npm run typecheck`, `npm test` (unit + integration + API) and the relevant Playwright spec.
2. For AI-affecting tasks, run the relevant eval suite and paste the headline metrics into PROGRESS.md.
3. Verify each acceptance criterion on the card, one by one, stating how it was verified.
4. Report: changed files, tests added, acceptance results, risks, follow-ups, any deviation from the playbook.
5. Update ARCHITECTURE.md and PROGRESS.md; commit with message `W<w>-<nn>: <task name>`.

Prompt template for each session: "Implement task W2-05 from docs/PLAYBOOK.md. Follow the session protocol in CLAUDE.md. Do not start other tasks."

Consistency safeguards

- A route-policy test fails when an endpoint lacks a declared role.

- ESLint's built-in no-restricted-imports rule forbids `modules/*` from importing `ai/llm` directly and `domain/` from importing I/O.
- Startup checks fail on config drift (taxonomy without a queue, prompt without a version).
- Review the diff yourself at every checkpoint; start a fresh session for each task rather than one long session.

13. Mandatory checkpoints

Do not start the next phase until every item at the checkpoint is true. Record the result in `docs/PROGRESS.md`.

Checkpoint	After	Must work before moving on
CP-1 Foundation	W1-06	`docker compose up` + seed works on a clean clone; all three roles log in; RBAC 401/403 tests green; config validation fails fast; lint/typecheck/tests green
CP-2 Retrieval	W1-12	Corpus and golden set validated; full corpus ingested idempotently; fused recall@5 ≥ 75%; forbidden-doc leakage 0; access-filter exhaustive tests green
CP-3 AI orchestration	W2-07 + W2-10	Graph compiles with all paths; each decision path covered by workflow tests; citation validator unit-tested; sensitive 100% and citation validity 100% on full eval; baseline decision accuracy recorded
CP-4 Employee flow	W2-09	Flows 1, 2, 3, 6 (offer step), 7 work in the browser; streaming TTFT measured; traces persisted for every turn; Playwright specs green
CP-5 Ticketing	W3-04	Flows 4, 5, 6 work; no ticket before confirmation; requester/queue/priority set by code; duplicate prompt works; interrupt resumes across requests
CP-6 Agent flow	W3-07	Flow 8 works; queue scoping and restricted visibility tests green; corrections stored and counted

Checkpoint	After	Must work before moving on
CP-7 Admin flow	W4-05	Flow 9 works: metrics, gaps, traces, document upload and re-index, eval run from UI
CP-8 Evaluation	W4-06	All section 8 targets met (or deviation ADR with plan); thresholds set by sweep; holdout reported; manual groundedness check recorded
CP-9 Final hardening	W4-10	Section 9 controls verified; NFR latency targets met; reset restores demo state; all nine flows pass in Playwright; two clean rehearsals; Final DoD checked

14. Scope protection

The contract defines the product; this list defines the guardrails. Claude Code must not build anything below. If it believes one is needed, it stops and writes a proposal stating the problem, why the existing design cannot solve it, the cost in days, and what gets cut to make room. Nothing is added without your explicit approval and an ADR.

Do Not Build

Category	Not allowed	Use instead
Architecture	Microservices, separate AI service, message brokers, event buses	Modular monolith: web + api
AI orchestration	Multi-agent systems, supervisor/worker agents, LLM-chosen routing, LLM tool binding, autonomous loops	One `StateGraph` with deterministic edges; graph-owned tool nodes
AI features	Additional LLM calls beyond P-1 – P-9, memory/personalisation beyond profile, sentiment analysis, auto-translation, auto-resolution of tickets, fine-tuning	The nine prompts
Integrations	ServiceNow, Jira, Workday, Zendesk, Slack, Teams, email, Confluence, SharePoint, calendar	Local tickets; ticket adapter interface only

Category	Not allowed	Use instead
Auth	Real SSO, OAuth providers, password management, self-registration, MFA	Seeded demo login
Infrastructure	Redis, Kubernetes, queues (BullMQ etc.), separate vector database, cloud deployment pipelines, Terraform	Docker Compose; in-process jobs; Atlas Vector Search
Data	Personal HR data (balances, salary), real documents, live sync	Synthetic corpus and profiles
Product	Notifications, SLA timers, mobile app, voice, multi-language, dashboards builder, exports, ticket attachments	Contract MVP only
Libraries	Mongoose, ORMs, state managers (Redux, Zustand), UI kits beyond Tailwind, chart libraries, LangSmith-only tracing as a requirement, extra validation libraries	Native driver, Zod, React state, Tailwind, own traces

Approved libraries (no approval needed): Next.js, React, Tailwind, Express, TypeScript, Zod, MongoDB Node driver, LangChain (core, Google GenAI, OpenAI-compatible, MongoDB, text splitters, PDF loader) and its PDF parsing dependency, LangGraph and its MongoDB checkpointer, jsonwebtoken, cookie-parser, cors, helmet, express-rate-limit, multer, pino, js-yaml, gray-matter, Jest, ts-jest, Supertest, Playwright, ESLint, Prettier. Anything else requires a one-line justification and approval.

OPTIONAL items (only after CP-9 or with explicit approval): employee ticket replies; similar resolved tickets for agents; eval run comparison view; eval in CI; a chart library for admin pages.

15. Final Definition of Done

The application is complete only when every item is checked.

Product

- [ ] All nine contract flows demonstrable end to end in the UI.
- [ ] Prototype banner visible on every page.
- [ ] Production gap list written (`docs/PRODUCTION_GAPS.md`).

- [ ] No Do Not Build item present; every deviation has an approved ADR.

Frontend

- [ ] All screens in section 10 implemented with loading, error and empty states.
- [ ] Streaming chat with stage indicator, citations, source drawer, withdrawn-answer handling.
- [ ] Role-based navigation and redirects; keyboard-navigable chat; readable contrast.

Backend

- [ ] Every endpoint in section 4 implemented with Zod request/response schemas and the error envelope.
- [ ] Route-policy test proves every route declares RBAC.
- [ ] Config validated at startup; fails fast on drift.
- [ ] Audit log written for every audited action.

Database

- [ ] All collections, indexes and constraints in section 3 exist via `npm run db:indexes`.
- [ ] Vector and text search indexes ready on a clean environment.
- [ ] `npm run reset` restores demo state; seed is idempotent; eval data isolated by `isEval`.

AI

- [ ] Single LangGraph `StateGraph` with deterministic edges; all nodes per section 5.
- [ ] Provider and models switchable between Gemini and OpenRouter by config only.
- [ ] All nine prompts versioned; every LLM call traced with prompt version, model, tokens, cost, latency.
- [ ] Every structured output Zod-validated with retry and documented fallback.
- [ ] Ticket creation only via human-in-the-loop interrupt and confirmation.
- [ ] Sensitive topics never receive AI advice.

RAG

- [ ] 30–40 document corpus with metadata, US/UK variants, superseded versions, tables, planted injection.
- [ ] Hybrid search with RRF and LLM rerank; access filter applied in the query and re-checked in code and on source view.
- [ ] Citations validated against retrieved chunks; numeric check active.

Evaluation

- [ ] Golden set of 80 cases with composition and tags per section 8, including holdout.
- [ ] Retrieval, full and judge suites run from the CLI and the admin page; results stored.
- [ ] Every section 8 target met; hard gates at 100% / 0.
- [ ] Thresholds chosen by sweep with ADR; manual groundedness spot check recorded.

Security

- [ ] JWT cookie, CORS allowlist, CSRF header, helmet, rate limits, body and upload limits active.
- [ ] Restricted tickets visible only to `hr_case` agents and admins; excluded from gaps and exports.
- [ ] PII redaction applied to traces, gaps and eval exports.
- [ ] Secrets only in env; `.env.example` complete; `npm audit` triaged.

Testing

- [ ] Jest unit tests for `domain/`, guards, retrieval, access rules, scorers, config (100% branch coverage on `domain/`).
- [ ] AI workflow tests with fake models for every graph path.
- [ ] Supertest API tests for every endpoint, including role matrix and ownership.
- [ ] Playwright specs for all nine flows, green twice in a row.

Performance

- [ ] TTFT p50 < 2.5 s, p90 < 4 s; full answer p90 < 10 s; ticket draft p90 < 6 s, measured and stored.
- [ ] 10 concurrent users without errors.
- [ ] Provider timeout produces a friendly, retryable error.

Documentation

- [ ] README: setup in ≤ 5 commands, configuration, architecture overview, AI/code responsibility split, eval results, security notes.
- [ ] ARCHITECTURE.md matches the code; DECISIONS.md complete; PROGRESS.md shows all tasks and checkpoints done.

Demo

- [ ] `docs/DEMO_SCRIPT.md` follows the contract's 8–10 minute script.
- [ ] Historic data populates dashboards; reset between rehearsals works.

- [ ] Two consecutive clean rehearsals; backup recording saved.